

DOCKER - CHALLENGE

1. Create a customized Docker image using a Dockerfile.

This is a basic Docker practical. We will create a custom **Nginx Docker image** using a Dockerfile, add our own HTML page, build the image, run a container, and verify it.

1. Objective

Create a customized Docker image using:

- nginx as the base image
- Custom index.html
- Dockerfile
- Port 80
- Custom Docker image
- Docker container

2. Check Docker Installation

First verify Docker:

```
docker --version
```

Example:

Docker version 28.x.x

Check Docker service:

```
systemctl status docker
```

If Docker is not running:

```
systemctl start docker
```

Enable Docker at boot:

```
systemctl enable docker
```

```
root@ip-172-31-30-131:~  
[root@ip-172-31-30-131 ~]# systemctl status docker  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: d>  
   Active: active (running) since Thu 2026-08-27 10:53:52 UTC; 9min ago  
 TriggeredBy: ● docker.socket  
    Docs: https://docs.docker.com  
 Process: 1930 ExecStartPre=/bin/mkdir -p /run/docker (code=exited, status=0>  
 Process: 1934 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (co>  
 Main PID: 1942 (dockerd)  
    Tasks: 9  
   Memory: 93.5M  
      CPU: 855ms  
   CGroup: /system.slice/docker.service  
           └─1942 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/cont>  
Aug 27 10:53:49 ip-172-31-30-131.ec2.internal systemd[1]: Starting docker.servi>  
Aug 27 10:53:50 ip-172-31-30-131.ec2.internal dockerd[1942]: time="2026-08-27T1>  
Aug 27 10:53:50 ip-172-31-30-131.ec2.internal dockerd[1942]: time="2026-08-27T1>  
Aug 27 10:53:50 ip-172-31-30-131.ec2.internal dockerd[1942]: time="2026-08-27T1>  
Aug 27 10:53:51 ip-172-31-30-131.ec2.internal dockerd[1942]: time="2026-08-27T1>  
Aug 27 10:53:51 ip-172-31-30-131.ec2.internal dockerd[1942]: time="2026-08-27T1>  
Aug 27 10:53:51 ip-172-31-30-131.ec2.internal dockerd[1942]: time="2026-08-27T1>  
Aug 27 10:53:51 ip-172-31-30-131.ec2.internal dockerd[1942]: time="2026-08-27T1>  
Aug 27 10:53:52 ip-172-31-30-131.ec2.internal dockerd[1942]: time="2026-08-27T1>
```

3. Create a Project Directory

Create a directory for the Docker project:

```
mkdir custom-nginx
```

Move into it:

```
cd custom-nginx
```

Check:

```
pwd
```

Expected:

```
/root/custom-nginx
```

```
root@ip-172-31-30-131:~/custom-nginx  
[root@ip-172-31-30-131 ~]# mkdir custom-nginx  
[root@ip-172-31-30-131 ~]# cd custom-nginx  
[root@ip-172-31-30-131 custom-nginx]# pwd  
/root/custom-nginx  
[root@ip-172-31-30-131 custom-nginx]# |
```

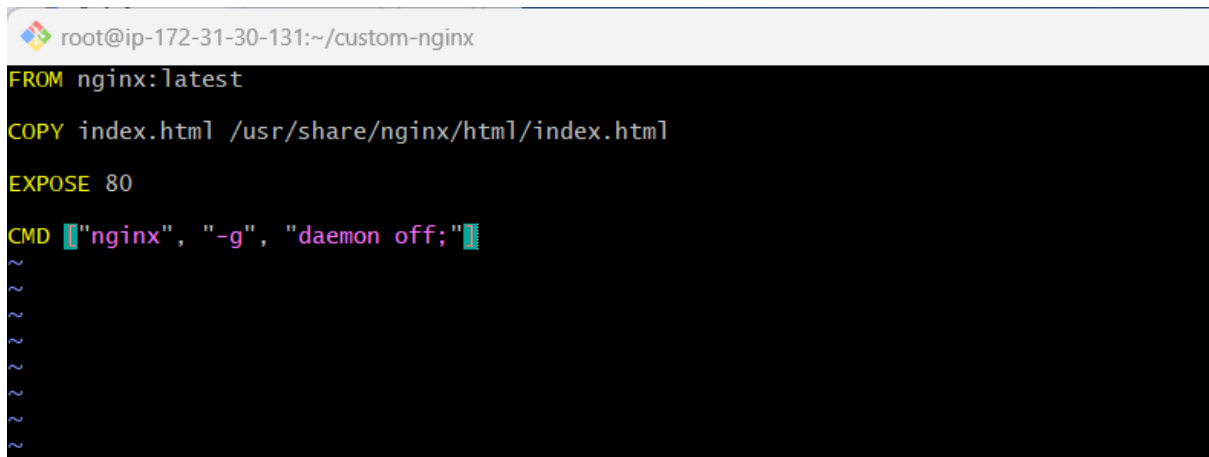
4. Create the Dockerfile

Create a file named exactly:

Dockerfile

Command:

vi Dockerfile

A terminal window with a dark background and light-colored text. The title bar shows 'root@ip-172-31-30-131:~/custom-nginx'. The terminal content shows a Dockerfile with the following lines: 'FROM nginx:latest', 'COPY index.html /usr/share/nginx/html/index.html', 'EXPOSE 80', and 'CMD ["nginx", "-g", "daemon off;"]'. There are several tilde characters (~) below the CMD line, indicating a scrollable history.

```
root@ip-172-31-30-131:~/custom-nginx
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
~
~
~
~
~
~
~
```

5. Understand the Dockerfile Line by Line

FROM

FROM nginx:latest

This specifies the **base image**.

Docker will use the official Nginx image as the starting point.

nginx:latest

means:

Image = nginx

Tag = latest

COPY

COPY index.html /usr/share/nginx/html/index.html

This copies our local HTML file into the Docker image.

Source:

index.html

Destination:

/usr/share/nginx/html/index.html

Nginx uses this directory to serve web content.

EXPOSE

EXPOSE 80

This documents that the application inside the container listens on port **80**.

Important: EXPOSE does **not** publish the port to the host.

Port publishing happens with:

`docker run -p 8080:80 ...`

CMD

`CMD ["nginx", "-g", "daemon off;"]`

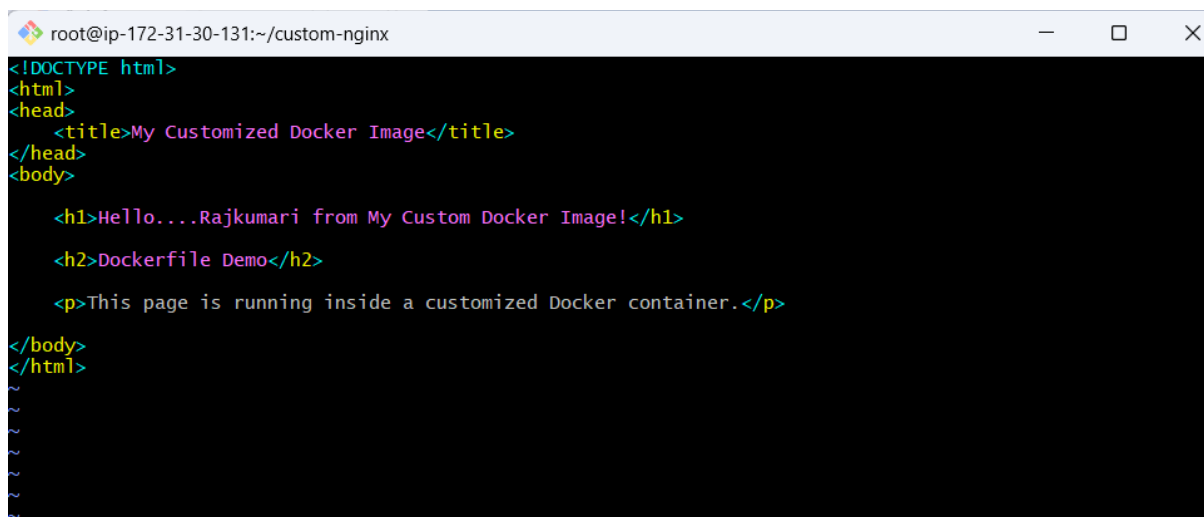
This starts Nginx in the foreground.

The container needs a foreground process to remain running.

6. Create the Custom HTML Page

Create:

`vi index.html`

A screenshot of a terminal window with a dark background and light-colored text. The window title is 'root@ip-172-31-30-131:~/custom-nginx'. The text shows the HTML code for 'index.html', including DOCTYPE, html, head, title, body, h1, h2, and p tags. The content is as follows:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Customized Docker Image</title>
</head>
<body>
  <h1>Hello....Rajkumari from My Custom Docker Image!</h1>
  <h2>Dockerfile Demo</h2>
  <p>This page is running inside a customized Docker container.</p>
</body>
</html>
```

7. Verify Project Files

Run:

```
ls -l
```

You should see:

```
Dockerfile
```

```
index.html
```

Your project structure is:

```
custom-nginx/
```

```
|
```

```
|— Dockerfile
```

```
└— index.html
```

8. Build the Customized Docker Image

Now build the image:

```
docker build -t my-custom-nginx:v1 .
```

Explanation

```
docker build
```

Builds a Docker image.

```
-t
```

Assigns a name and tag.

```
my-custom-nginx:v1
```

Image name and tag.

```
.
```

The current directory is used as the **build context**.

9. Observe the Build

You should see something similar to:

```
root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# docker build -t my-custom-nginx:v1 .
[+] Building 0.6s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 212B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:latest  0.3s
=> [auth] library/nginx:pull token for registry-1.docker.io     0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 370B                                  0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:latest@sha256:b34848eff6db786b6b1282d3a9c3fd0b5563dfb 0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html      0.1s
=> exporting to image                                           0.0s
=> => exporting layers                                           0.0s
=> => writing image sha256:22b7519cba9450d289de98270c4d5222c1f3e36b1b324cf9ae6c48e9e9404fde 0.0s
=> => naming to docker.io/library/my-custom-nginx:v1           0.0s
[root@ip-172-31-30-131 custom-nginx]#
```

10. Verify the Docker Image

Run: docker images

```
root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# docker images
REPOSITORY              TAG         IMAGE ID      CREATED      SIZE
my-custom-nginx         v1         22b7519cba94  58 seconds ago  162MB
jenkins-docker-demo     v1         04f7a31a23ca  3 hours ago   162MB
rajkumari2010/jenkins-docker-demo  1         04f7a31a23ca  3 hours ago   162MB
rajkumari2010/jenkins-docker-demo  2         04f7a31a23ca  3 hours ago   162MB
rajkumari2010/jenkins-docker-demo  3         04f7a31a23ca  3 hours ago   162MB
rajkumari2010/jenkins-docker-demo  4         04f7a31a23ca  3 hours ago   162MB
rajkumari2010/jenkins-docker-demo  5         04f7a31a23ca  3 hours ago   162MB
docker-scan-demo        v2         071c993a79a7  5 hours ago   162MB
571833381790.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo v1         34b3ffb37895  6 hours ago   162MB
docker-scan-demo        v1         34b3ffb37895  6 hours ago   162MB
dockerhub-scan-demo     v1         34b3ffb37895  6 hours ago   162MB
```

11. Inspect the Image

You can inspect the image:

docker inspect my-custom-nginx:v1

You can also check its history:

docker history my-custom-nginx:v1

This is useful for understanding the image layers.

You should see a layer corresponding to:

12. Run a Container From Your Customized Image

Run:

```
docker run -d \  
    --name custom-nginx-container \  
    -p 8080:80 \  
    my-custom-nginx:v1
```

Understand the port mapping

-p 8080:80

means:

Host port 8080

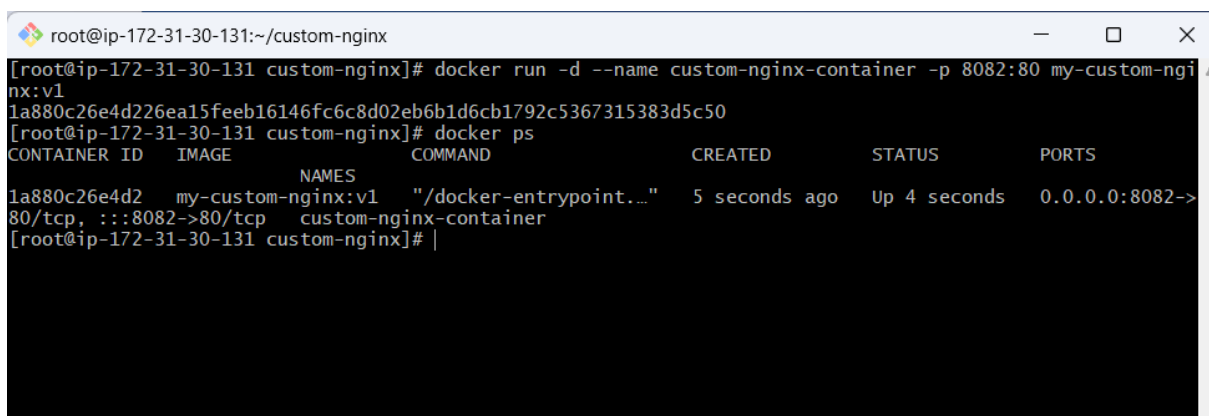
↓

Container port 80

So:

http://<server-ip>:8080

will reach Nginx inside the container

A terminal window titled 'root@ip-172-31-30-131:~/custom-nginx' with standard window controls. The terminal shows the execution of 'docker run' to start a container named 'custom-nginx-container' with port mapping '-p 8082:80'. The container ID '1a880c26e4d226ea15feeb16146fc6c8d02eb6b1d6cb1792c5367315383d5c50' is displayed. Then, 'docker ps' is run, showing a table of running containers. The table has columns: CONTAINER ID, IMAGE, NAMES, COMMAND, CREATED, STATUS, and PORTS. One container is listed: '1a880c26e4d2' with image 'my-custom-nginx:v1', name 'custom-nginx-container', command '/docker-entrypoint...', created '5 seconds ago', status 'Up 4 seconds', and ports '0.0.0.0:8082->80/tcp, :::8082->80/tcp'.

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
1a880c26e4d2	my-custom-nginx:v1	custom-nginx-container	"/docker-entrypoint..."	5 seconds ago	Up 4 seconds	0.0.0.0:8082->80/tcp, :::8082->80/tcp

14. Test From Your Browser

If this is an EC2 instance, use:

http://<EC2-Public-IP>:8080



Hello....Rajkumari from My Custom Docker Image!

Dockerfile Demo

This page is running inside a customized Docker container.

6. Verify the Customized File Inside the Container

Find the container:

```
docker ps
```

Then execute a command inside it:

```
docker exec -it custom-nginx-container /bin/bash
```

If Bash isn't available, try:

```
docker exec -it custom-nginx-container /bin/sh
```

Then:

```
cat /usr/share/nginx/html/index.html
```

You should see your customized HTML.

Exit:

2. Push the same image to Docker Hub.

Push the Same Docker Image to Docker Hub

Since you already created the customized image:

```
my-custom-nginx:v1
```

we can push **the same image** to Docker Hub without rebuilding it.

The process is:

Existing Local Image

|



Docker Tag



Docker Hub Login



Docker Push



Docker Hub

1. Verify the Existing Image

First check your local images:

`docker images`

You should see something similar to:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-custom-nginx	v1	abc123456789	5 minutes ago	...

The important part is:

`my-custom-nginx:v1`

```
root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my-custom-nginx     v1                 22b7519cba94       11 minutes ago     162MB
jenkins-docker-demo v1                 04f7a31a23ca       4 hours ago        162MB
rajkumari2010/jenkins-docker-demo 1                 04f7a31a23ca       4 hours ago        162MB
rajkumari2010/jenkins-docker-demo 2                 04f7a31a23ca       4 hours ago        162MB
rajkumari2010/jenkins-docker-demo 3                 04f7a31a23ca       4 hours ago        162MB
rajkumari2010/jenkins-docker-demo 4                 04f7a31a23ca       4 hours ago        162MB
rajkumari2010/jenkins-docker-demo 5                 04f7a31a23ca       4 hours ago        162MB
```

2. Create a Docker Hub Repository

Log in to Docker Hub:

[Docker Hub](#)

Go to:

Docker Hub



Repositories



Create repository

Create a repository named:

my-custom-nginx

For example, if your Docker Hub username is:

rajkumari

your repository will be:

rajkumari/my-custom-nginx



[Repositories](#) / [my-custom-nginx](#) / [General](#)
Using 0 of 1 private repositories.

rajkumari2010/my-custom-nginx 
Created less than a minute ago · ☆0 · ↓0

[Add a description](#)  

[Add a category](#)  

Docker commands [Public view](#)

To push a new tag to this repository:

```
docker push rajkumari2010/my-custom-nginx:tagname
```

[General](#) [Tags](#) [Image Management](#) [Collaborators](#) [Webhooks](#) [Settings](#)

Tags  INCOMPLETE  DOCKER SCOUT INACTIVE [Activate](#)
Pushed images appear here.

3. Login to Docker Hub

On your Docker server:

```
docker login
```

Enter:

Username: <your-dockerhub-username>

Password: <your-Docker-Hub-PAT>

For example:

Username: rajkumari

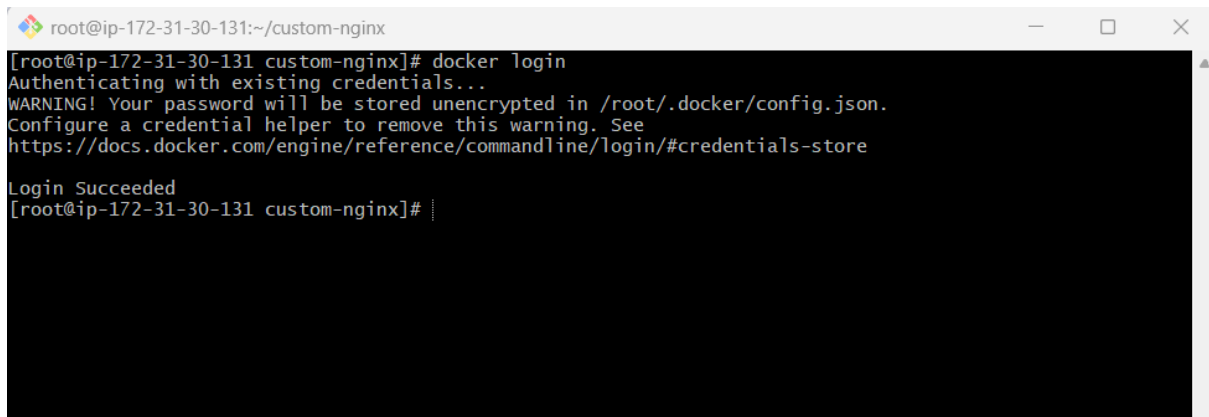
Password: *****

Expected:

Login Succeeded

Important

Use a **Docker Hub Personal Access Token (PAT)** instead of putting your Docker Hub account password into scripts.

A terminal window with a black background and white text. The title bar shows 'root@ip-172-31-30-131:~/custom-nginx'. The terminal output shows the command 'docker login' being executed. It displays 'Authenticating with existing credentials...', a warning about unencrypted passwords, a URL for credential helpers, and finally 'Login Succeeded'. The prompt returns to '[root@ip-172-31-30-131 custom-nginx]# |'.

4. Tag the Existing Image

This is the most important step.

Your current image is:

my-custom-nginx:v1

Docker Hub requires the image name to include your Docker Hub username.

Run:

```
docker tag my-custom-nginx:v1 \  
<dockerhub-username>/my-custom-nginx:v1
```

For example:

```
docker tag my-custom-nginx:v1 \  
rajkumari/my-custom-nginx:v1
```

What happened?

You did **not** create another image.

You simply created another **tag/reference** pointing to the same image.

my-custom-nginx:v1

|

| same IMAGE ID

↓

rajkumari/my-custom-nginx:v1

```
root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# docker tag my-custom-nginx:v1 rajkumari2010/my-custom-nginx:v1
[root@ip-172-31-30-131 custom-nginx]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED
my-custom-nginx	v1	22b7519cba94	17 minutes ago
rajkumari2010/my-custom-nginx	v1	22b7519cba94	17 minutes ago
jenkins-docker-demo	v1	04f7a31a23ca	4 hours ago
rajkumari2010/jenkins-docker-demo	1	04f7a31a23ca	4 hours ago
rajkumari2010/jenkins-docker-demo	2	04f7a31a23ca	4 hours ago
rajkumari2010/jenkins-docker-demo	3	04f7a31a23ca	4 hours ago
rajkumari2010/jenkins-docker-demo	4	04f7a31a23ca	4 hours ago

6. Push the Image to Docker Hub

Now run:

```
docker push rajkumari/my-custom-nginx:v1
```

Replace rajkumari with your Docker Hub username.

```
root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# docker push rajkumari2010/my-custom-nginx:v1
The push refers to repository [docker.io/rajkumari2010/my-custom-nginx]
afb8ed98c0ec: Pushed
a8fae4102358: Mounted from rajkumari2010/jenkins-docker-demo
b0020123c41b: Mounted from rajkumari2010/jenkins-docker-demo
f29840e7d6e5: Mounted from rajkumari2010/jenkins-docker-demo
f109061e6486: Mounted from rajkumari2010/jenkins-docker-demo
e8fb5de3ef79: Mounted from rajkumari2010/jenkins-docker-demo
070ef859f070: Mounted from rajkumari2010/jenkins-docker-demo
411a86676185: Mounted from rajkumari2010/jenkins-docker-demo
v1: digest: sha256:3c18c1d829ab9e9b41f77b2772058735b9d7c2bf53d2592914c7df9c2557b520 size: 1985
[root@ip-172-31-30-131 custom-nginx]#
```

7. Verify in Docker Hub

Go to:

[Docker Hub Repositories](#)

Navigate to:

Repositories



my-custom-nginx



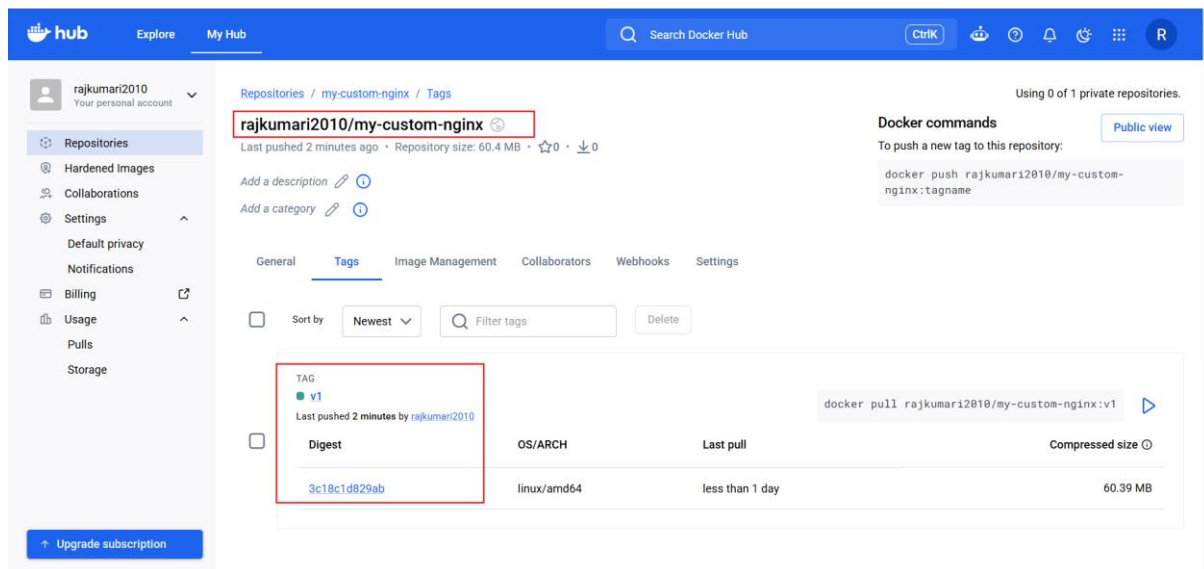
Tags

You should see:

v1

Your image is now:

rajkumari/my-custom-nginx:v1



3. Push the same image to Amazon ECR.

1. Verify the Existing Image

First, check your local Docker images:

docker images

You should see:

REPOSITORY	TAG	IMAGE ID
------------	-----	----------

my-custom-nginx	v1	abc123456789
-----------------	----	--------------

If you already tagged it for Docker Hub, you may see:

REPOSITORY	TAG	IMAGE ID
------------	-----	----------

my-custom-nginx	v1	abc123456789
-----------------	----	--------------

rajkumari/my-custom-nginx	v1	abc123456789
---------------------------	----	--------------

The important point is that the **IMAGE ID** is the same.

2. Verify AWS CLI

Run:

```
aws --version
```

Then check your AWS identity:

```
aws sts get-caller-identity
```

3. Set Your AWS Region

For this example, we'll use:

```
us-east-1
```

You can verify your current AWS region:

```
aws configure get region
```

If required:

```
aws configure set region us-east-1
```

4. Create an ECR Repository

Create a repository:

```
aws ecr create-repository \  
    --repository-name my-custom-nginx \  
    --region us-east-1
```

Expected response will contain something like:

repositoryUri:

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx
```

Your ECR repository URL is:

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx
```

Replace 123456789012 with your AWS Account ID.

```

root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.18.39-79.141.amzn2023.x86_64 source/x86_64.amzn.2023
[root@ip-172-31-30-131 custom-nginx]# aws sts get-caller-identity
{
  "UserId": "AROAYKI7M6OP0WVOJDCRB:i-092add6ce1e26dacf",
  "Account": "571833381790",
  "Arn": "arn:aws:sts::571833381790:assumed-role/EC2-ECR-Role/i-092add6ce1e26dacf"
}
[root@ip-172-31-30-131 custom-nginx]# aws ecr create-repository \
--repository-name my-custom-nginx \
--region us-east-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:571833381790:repository/my-custom-nginx",
    "registryId": "571833381790",
    "repositoryName": "my-custom-nginx",
    "repositoryUri": "571833381790.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx",
    "createdAt": "2026-08-27T11:46:31.472000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
[root@ip-172-31-30-131 custom-nginx]# |

```

5. Verify the ECR Repository

Run:

```

aws ecr describe-repositories \
  --repository-names my-custom-nginx \
  --region us-east-1

```

You should see:

repositoryName: my-custom-nginx

repositoryUri: 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx


```

root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# aws ecr describe-repositories \
--repository-names my-custom-nginx \
--region us-east-1
{
  "repositories": [
    {
      "repositoryArn": "arn:aws:ecr:us-east-1:571833381790:repository/my-custom-nginx",
      "registryId": "571833381790",
      "repositoryName": "my-custom-nginx",
      "repositoryUri": "571833381790.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx",
      "createdAt": "2026-08-27T11:46:31.472000+00:00",
      "imageTagMutability": "MUTABLE",
      "imageScanningConfiguration": {
        "scanOnPush": false
      },
      "encryptionConfiguration": {
        "encryptionType": "AES256"
      }
    }
  ]
}
[root@ip-172-31-30-131 custom-nginx]#

```

6. Login to Amazon ECR

This is an important step.

Run:

```

[root@ip-172-31-30-131 custom-nginx]# aws ecr get-login-password --region us-east-1 | \
docker login \
--username AWS \
--password-stdin \
571833381790.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-30-131 custom-nginx]#

```

7. Tag the Same Image for ECR

Your existing image is:

my-custom-nginx:v1

Tag it with your ECR repository URL:

docker tag my-custom-nginx:v1 \

123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:v1

Replace:

123456789012

with your AWS Account ID.

8. Verify the Image Tags

Run:

docker images

```
root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# docker tag my-custom-nginx:v1 571833381790.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:v1
[root@ip-172-31-30-131 custom-nginx]# docker images
-bash: docker: command not found
[root@ip-172-31-30-131 custom-nginx]# docker images
REPOSITORY                                TAG      IMAGE ID      CREATED
SIZE
571833381790.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx  v1       22b7519cba94  37 minutes ago
162MB
my-custom-nginx                                v1       22b7519cba94  37 minutes ago
162MB
```

9. Push the Image to ECR

Now push: `docker push <aws account ID>.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:v1`

```
root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# docker push \
571833381790.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:v1
The push refers to repository [571833381790.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx]
afb8ed98c0ec: Pushed
a8fae4102358: Pushed
b0020123c41b: Pushed
f29840e7d6e5: Pushed
f109061e6486: Pushed
e8fb5de3ef79: Pushed
070ef859f070: Pushed
411a86676185: Pushed
v1: digest: sha256:3c95fb12719e049f9b4780aaee15a827d50b5f26cb4aa78767c77dc87600d197 size: 1985
[root@ip-172-31-30-131 custom-nginx]#
```

10. Verify the Image in ECR

Run:

```
aws ecr list-images \
    --repository-name my-custom-nginx \
    --region us-east-1
```

11. Get Detailed Image Information

Run:

```
aws ecr describe-images \
    --repository-name my-custom-nginx \
    --image-ids imageTag=v1 \
    --region us-east-1
```

```

root@ip-172-31-30-131:~/custom-nginx
[root@ip-172-31-30-131 custom-nginx]# aws ecr list-images \
--repository-name my-custom-nginx \
--region us-east-1
{
  "imageIds": [
    {
      "imageDigest": "sha256:3c95fb12719e049f9b4780aaee15a827d50b5f26cb4aa78767c77dc87600d197",
      "imageTag": "v1"
    }
  ]
}
[root@ip-172-31-30-131 custom-nginx]# aws ecr describe-images \
--repository-name my-custom-nginx \
--image-ids imageTag=v1 \
--region us-east-1
{
  "imageDetails": [
    {
      "registryId": "571833381790",
      "repositoryName": "my-custom-nginx",
      "imageDigest": "sha256:3c95fb12719e049f9b4780aaee15a827d50b5f26cb4aa78767c77dc87600d197",
      "imageTags": [
        "v1"
      ],
      "imageSizeInBytes": 64341228,
      "imagePushedAt": "2026-08-27T11:55:54.033000+00:00",
      "imageManifestMediaType": "application/vnd.docker.distribution.manifest.v2+json",
      "artifactMediaType": "application/vnd.docker.container.image.v1+json",
      "imageStatus": "ACTIVE"
    }
  ]
}
[root@ip-172-31-30-131 custom-nginx]#

```

12. Verify in AWS Console

Open:

AWS Console

↓

Amazon ECR

↓

Repositories

↓

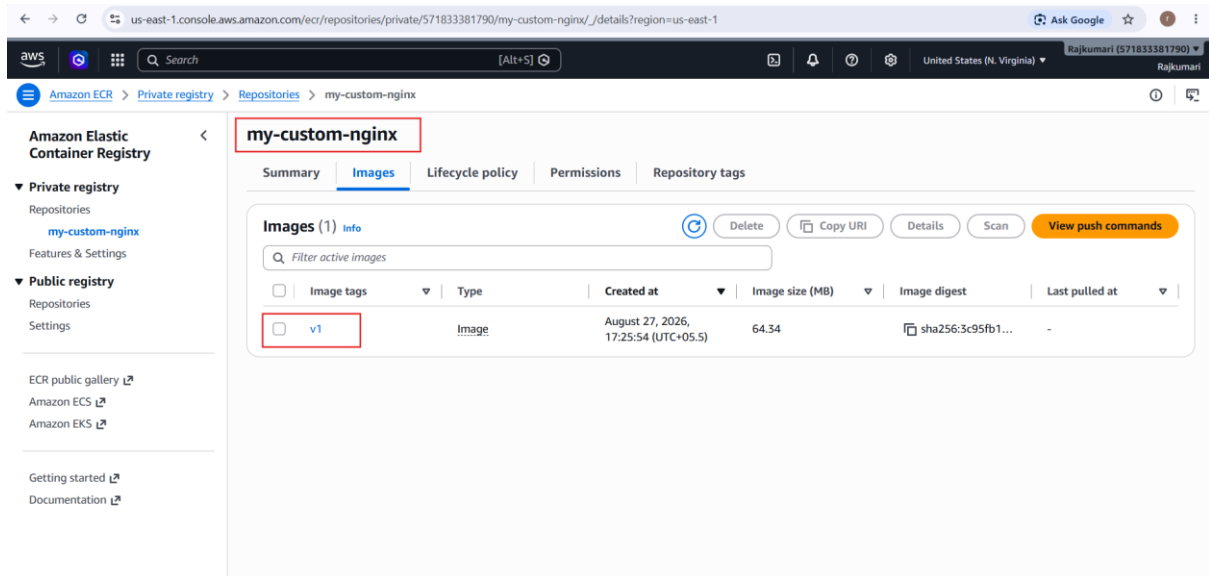
my-custom-nginx

You should see:

Image tag

v1

Click the image to view its details.



4. Provision one EC2 instance using Terraform and install Jenkins.

1. Prerequisites

You need:

- AWS account
- AWS IAM permissions to create EC2/security groups
- AWS CLI
- Terraform
- SSH client
- An EC2 key pair

Check Terraform:

```
terraform --version
```

Check AWS CLI:

```
aws --version
```

Check AWS credentials:

```
aws sts get-caller-identity
```

If you're working from an AWS EC2 server that already has an IAM role, the AWS CLI can use that role.

```
root@ip-172-31-30-131:~  
Dependencies resolved.  
Nothing to do.  
Complete!  
[root@ip-172-31-30-131 ~]# sudo dnf config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo  
Adding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo  
[root@ip-172-31-30-131 ~]# sudo dnf install -y terraform  
Hashicorp Stable - x86_64 19 MB/s | 2.1 MB 00:00  
Last metadata expiration check: 0:00:01 ago on Thu Aug 27 12:06:04 2026.  
Dependencies resolved.  
=====
```

Package	Architecture	Version	Repository	Size
Installing: terraform	x86_64	1.16.0-1	hashicorp	34 M

```
=====
```

Transaction Summary			
Install 1 Package			
Total download size: 34 M			
Installed size: 115 M			
Downloading Packages:			
terraform-1.16.0-1.x86_64.rpm		140 MB/s 34 MB	00:00

Total		137 MB/s 34 MB	00:00
Hashicorp Stable - x86_64		179 kB/s 3.9 kB	00:00

```
=====
```

Importing GPG key 0xA621E701:
Userid : "HashiCorp Security (HashiCorp Package Signing) <security+packaging@hashicorp.com>"
Fingerprint: 798A EC65 4E5C 1542 8C8E 42EE AA16 FCBC A621 E701
From : https://rpm.releases.hashicorp.com/gpg
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :
Installing : terraform-1.16.0-1.x86_64 1/1
Verifying : terraform-1.16.0-1.x86_64 1/1
=====

WARNING:
A newer release of "Amazon Linux" is available.

Available Versions:

Version 2023.12.20260817:
Run the following command to upgrade to 2023.12.20260817:

dnf upgrade --releasever=2023.12.20260817

Release notes:
<https://docs.aws.amazon.com/linux/a12023/release-notes/re1notes-2023.12.20260817.html>
=====

Installed:
terraform-1.16.0-1.x86_64

2. Create a Terraform Project

Create a directory:

```
mkdir terraform-jenkins
```

```
cd terraform-jenkins
```

Check:

```
pwd
```

Project structure will eventually be:

```
terraform-jenkins/
```

```
|
```

```
|— provider.tf
```

|— variables.tf

|— main.tf

|— outputs.tf

|— userdata.sh

3. Create provider.tf

Create:

vi provider.tf

A screenshot of a terminal window with a black background and white text. The terminal prompt is 'root@ip-172-31-30-131:~/terraform-jenkins'. The user has opened a file named 'provider.tf' in a text editor. The file contains Terraform configuration for the AWS provider. The configuration includes a 'terraform' block with 'required_providers' and 'required_version' settings, and a 'provider' block for 'aws' that sets the 'region' to 'var.aws_region'. The terminal shows the file being edited with a cursor at the end of the line 'region = var.aws_region'.

```
root@ip-172-31-30-131:~/terraform-jenkins
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }

  required_version = ">= 1.5.0"
}

provider "aws" {
  region = var.aws_region
}
```

Explanation

The provider tells Terraform that we are using AWS.

provider "aws"

The region is provided through:

var.aws_region

4. Create variables.tf

Create:

vi variables.tf

```
root@ip-172-31-30-131:~/terraform-jenkins
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "key_name" {
  description = "Existing EC2 key pair name"
  type        = string
}
```

Here:

aws_region → AWS region

instance_type → EC2 instance type

key_name → EC2 key pai

5. Create userdata.sh

This script will automatically install Jenkins after EC2 is created.

Create:

```
root@ip-172-31-30-131:~/terraform-jenkins
#!/bin/bash

set -e

# Update packages
dnf update -y

# Install Java 17
dnf install -y java-17-amazon-corretto

# Add Jenkins repository
curl -fsSL https://pkg.jenkins.io/redhat-stable/jenkins.io-2026.key \
  -o /etc/pki/rpm-gpg/jenkins.io-2026.key

cat > /etc/yum.repos.d/jenkins.repo <<'EOF'
[jenkins]
name=Jenkins
baseurl=https://pkg.jenkins.io/redhat-stable
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/jenkins.io-2026.key
EOF

# Install Jenkins
dnf install -y jenkins

# Start Jenkins
systemctl enable jenkins
systemctl start jenkins|
~
~
~
~
~
~
```

Important

Jenkins repository keys and package URLs can change over time. If the current Jenkins repository rejects the key shown above, use the current installation instructions from the official Jenkins documentation rather than blindly reusing an old key.

6. Create main.tf

Create:


```

root@ip-172-31-30-131:~/terraform-jenkins
❯ terra "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
  filter {
    name     = "state"
    values   = ["available"]
  }
}

resource "aws_security_group" "jenkins_sg" {
  name        = "jenkins-security-group"
  description = "Security group for Jenkins"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Jenkins"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "jenkins-security-group"
  }
}

resource "aws_instance" "jenkins" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type
}

```

Important: AMI ID

The AMI ID above is **region-specific** and can become outdated.

For your own environment, use an Amazon Linux 2023 AMI available in your selected AWS region.

You can find it from:

AWS Console

↓

EC2

↓

AMIs

↓

Amazon Linux

Or use a Terraform data source instead of hardcoding an AMI ID.

9. Create outputs.tf

Create:

vi outputs.tf

```
root@ip-172-31-30-131:~/terraform-jenkins
output "instance_id" {
  description = "Jenkins EC2 instance ID"
  value       = aws_instance.jenkins.id
}

output "public_ip" {
  description = "Jenkins EC2 public IP"
  value       = aws_instance.jenkins.public_ip
}

output "jenkins_url" {
  description = "Jenkins URL"
  value       = "http://${aws_instance.jenkins.public_ip}:8080"
}

~
~
~
~
~
~
```

10. Your Final Project Structure

Run:

ls -l

You should have:

terraform-jenkins/

|

├── main.tf

├── provider.tf

├── variables.tf

├── outputs.tf

└── userdata.sh

```
root@ip-172-31-30-131:~/terraform-jenkins
[root@ip-172-31-30-131 terraform-jenkins]# vi main.tf
[root@ip-172-31-30-131 terraform-jenkins]# vi outputs.tf
[root@ip-172-31-30-131 terraform-jenkins]# ls -l
total 20
-rw-r--r--. 1 root root 1120 Aug 27 12:27 main.tf
-rw-r--r--. 1 root root 338 Aug 27 12:27 outputs.tf
-rw-r--r--. 1 root root 170 Aug 27 12:09 provider.tf
-rw-r--r--. 1 root root 538 Aug 27 12:12 userdata.sh
-rw-r--r--. 1 root root 315 Aug 27 12:11 variables.tf
[root@ip-172-31-30-131 terraform-jenkins]# s|
```

11. Initialize Terraform

Run:

```
terraform init
```

Expected:

Initializing the backend...

Initializing provider plugins...

Terraform has been successfully initialized!

What does terraform init do?

It downloads the AWS provider and initializes the Terraform working directory

```
root@ip-172-31-30-131:~/terraform-jenkins
[root@ip-172-31-30-131 terraform-jenkins]# terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.62.0...
- Installed hashicorp/aws v6.62.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@ip-172-31-30-131 terraform-jenkins]# |
```

```
root@ip-172-31-30-131:~/terraform-jenkins
aws_region      = "us-east-1"
instance_type   = "t3.micro"
key_name        = "server-2"
~
~
~
~
~
~
~
~
```

2. Format Terraform Files

Run:

```
terraform fmt
```

This formats your .tf files.

13. Validate Configuration

Run:

```
terraform validate
```

Expected:

Success! The configuration is valid.

If there is a syntax problem, Terraform will tell you the file and line.

```

root@ip-172-31-30-131:~/terraform-jenkins
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.62.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@ip-172-31-30-131 terraform-jenkins]# aws ec2 describe-key-pairs --query 'K
eyPairs[*].KeyName' --output table

An error occurred (UnauthorizedOperation) when calling the DescribeKeyPairs oper
ation: You are not authorized to perform this operation. User: arn:aws:sts::5718
33381790:assumed-role/EC2-ECR-Role/i-092add6ce1e26dacf is not authorized to perf
orm: ec2:DescribeKeyPairs because no identity-based policy allows the ec2:Descri
beKeyPairs action
[root@ip-172-31-30-131 terraform-jenkins]# aws ec2 describe-key-pairs --query 'K
eyPairs[*].KeyName' --output table
-----
|DescribeKeyPairs|
+-----+
| server-2       |
+-----+
[root@ip-172-31-30-131 terraform-jenkins]# vi terraform.tfvars
[root@ip-172-31-30-131 terraform-jenkins]# cat terraform.tfvars
aws_region      = "us-east-1"
instance_type   = "t3.micro"
key_name        = "server-2"
[root@ip-172-31-30-131 terraform-jenkins]# cat variables.tf
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "key_name" {
  description = "Existing EC2 key pair name"
  type        = string
}
[root@ip-172-31-30-131 terraform-jenkins]# vi variables.tf
[root@ip-172-31-30-131 terraform-jenkins]# terraform fmt
[root@ip-172-31-30-131 terraform-jenkins]# terraform validate
Success! The configuration is valid.
[root@ip-172-31-30-131 terraform-jenkins]# |

```

14. Create a Terraform Plan

You need to provide your EC2 key pair name.

For example, if your AWS key pair is:

my-ec2-key

Run:

```
terraform plan -var="key_name=my-ec2-key"
```

Terraform will show what it intends to create.

You should see resources similar to:

Plan: 2 to add, 0 to change, 0 to destroy.

The resources are:

1. Security Group
2. EC2 Instance

```

root@ip-172-31-30-131:~/terraform-jenkins
[root@ip-172-31-30-131 terraform-jenkins]# aws ec2 describe-images \
--owners amazon \
--filters "Name=name,Values=al2023-ami-*-x86_64" \
--query 'Images | sort_by(@, &CreationDate)[-1].[ImageId,Name,CreationDate]' \
--output table
+-----+-----+-----+
| DescribeImages |
+-----+-----+-----+
| ami-0fe74bfcad4fd6bd2 |
| al2023-ami-ecs-hvm-2023.0.20260820-kernel-6.1-x86_64 |
| 2026-08-21T19:35:04.000Z |
+-----+-----+-----+
[root@ip-172-31-30-131 terraform-jenkins]# vi main.tf
[root@ip-172-31-30-131 terraform-jenkins]# vi main.tf
[root@ip-172-31-30-131 terraform-jenkins]# terraform fmt
[root@ip-172-31-30-131 terraform-jenkins]# terraform validate
Success! The configuration is valid.

[root@ip-172-31-30-131 terraform-jenkins]# aws ec2 describe-key-pairs \
--query 'KeyPairs[*].KeyName' \
--output table
+-----+
| DescribeKeyPairs |
+-----+
| server-2 |
+-----+
[root@ip-172-31-30-131 terraform-jenkins]# cat terraform.tfvars
aws_region = "us-east-1"
instance_type = "t3.micro"
key_name = "server-2"
[root@ip-172-31-30-131 terraform-jenkins]# terraform plan
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-0fe74bfcad4fd6bd2]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.jenkins will be created
+ resource "aws_instance" "jenkins" {
+   ami = "ami-0fe74bfcad4fd6bd2"
+   arn = (known after apply)
+   associate_public_ip_address = (known after apply)
+   availability_zone = (known after apply)
+   disable_api_stop = (known after apply)
+   disable_api_termination = (known after apply)
+   ebs_optimized = (known after apply)
+   enable_primary_ipv6 = (known after apply)
+   force_destroy = false
+   get_password_data = false
+   host_id = (known after apply)
+   host_resource_group_arn = (known after apply)
+   iam_instance_profile = (known after apply)
+   id = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)

```

```

root@ip-172-31-30-131:~/terraform-jenkins
+ enable_primary_ipv6 = (known after apply)
+ force_destroy       = false
+ get_password_data   = false
+ host_id             = (known after apply)
+ host_resource_group_arn = (known after apply)
+ iam_instance_profile = (known after apply)
+ id                 = (known after apply)
+ instance_initiated_shutdown_behavior = (known after apply)
+ instance_lifecycle  = (known after apply)
+ instance_state      = (known after apply)
+ instance_type       = "t3.micro"
+ ipv6_address_count  = (known after apply)
+ ipv6_addresses      = (known after apply)
+ key_name            = "server-2"
+ monitoring          = (known after apply)
+ outpost_arn         = (known after apply)
+ password_data       = (known after apply)
+ placement_group     = (known after apply)
+ placement_group_id  = (known after apply)
+ placement_partition_number = (known after apply)
+ primary_network_interface_id = (known after apply)
+ private_dns         = (known after apply)
+ private_ip          = (known after apply)
+ public_dns          = (known after apply)
+ public_ip           = (known after apply)
+ region              = "us-east-1"
+ secondary_private_ips = (known after apply)
+ security_groups      = (known after apply)
+ source_dest_check    = true
+ spot_instance_request_id = (known after apply)
+ subnet_id           = (known after apply)
+ tags                = {
+   + "Name" = "Jenkins-Terraform"
+ }
+ tags_all            = {
+   + "Name" = "Jenkins-Terraform"
+ }
+ tenancy              = (known after apply)
+ user_data            = <<-EOT
    #!/bin/bash

    set -e

    # Update packages
    dnf update -y

    # Install Java 17
    dnf install -y java-17-amazon-corretto

    # Add Jenkins repository
    curl -fsSL https://pkg.jenkins.io/redhat-stable/jenkins.io-2026.key
    -o /etc/pki/rpm-gpg/jenkins.io-2026.key

    cat > /etc/yum.repos.d/jenkins.repo <<'EOF'
    [jenkins]
    name=Jenkins
    baseurl=https://pkg.jenkins.io/redhat-stable

```

Step 5 — Verify the IAM change

After attaching the EC2 policy, wait a few seconds and run:

```
aws sts get-caller-identity
```

Then test whether your role can describe EC2 resources:

```
aws ec2 describe-security-groups --region us-east-1
```

If this returns security-group information rather than AccessDenied, that's a good sign.

Step 6 — Run Terraform Plan Again

Go back to:

```
cd terraform-jenkins
```

Run:

```
terraform plan -var="key_name=server-2"
```

You should see:

Plan: 2 to add, 0 to change, 0 to destroy.

The two resources are:

aws_security_group.jenkins_sg

aws_instance.jenkins

```
root@ip-172-31-30-131:~/terraform-jenkins
+ description      = "SSH"
+ from_port        = 22
+ ipv6_cidr_blocks = []
+ prefix_list_ids  = []
+ protocol         = "tcp"
+ security_groups  = []
+ self             = false
+ to_port          = 22
},
]
name              = "jenkins-security-group"
+ name_prefix      = (known after apply)
~ owner_id         = "571833381790" -> (known after apply)
tags              = {
  "Name" = "jenkins-security-group"
}
~ vpc_id           = "vpc-083187674976c0db2" -> (known after apply)
# (5 unchanged attributes hidden)
}

Plan: 2 to add, 0 to change, 1 to destroy.

Changes to Outputs:
+ instance_id = (known after apply)
+ jenkins_url = (known after apply)
+ public_ip   = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.jenkins_sg: Destroying... [id=sg-04b74d43feefa0e28]
aws_security_group.jenkins_sg: Destruction complete after 0s
aws_security_group.jenkins_sg: Creating...
aws_security_group.jenkins_sg: Creation complete after 3s [id=sg-0989df5cefdc1a624]
aws_instance.jenkins: Creating...
aws_instance.jenkins: Still creating... [00m10s elapsed]
aws_instance.jenkins: Creation complete after 13s [id=i-09f0e98400efc3011]

Apply complete! Resources: 2 added, 0 changed, 1 destroyed.

Outputs:
instance_id = "i-09f0e98400efc3011"
jenkins_url = "http://52.91.155.223:8080"
public_ip   = "52.91.155.223"
[root@ip-172-31-30-131 terraform-jenkins]# |
```

Step 7 — Apply Again

Now run:


```
terraform apply -var="key_name=server-2"
```

When prompted:

Do you want to perform these actions?

enter:

yes

This time Terraform should create:

Security Group



EC2 Instance



User Data



Java 17



Jenkins

Step 8 — Check Terraform Output

After successful completion:

```
terraform output
```

You should get something similar to:

```
instance_id = "i-0123456789abcdef"
```

```
jenkins_url = "http://54.xx.xx.xx:8080"
```

```
public_ip = "54.xx.xx.xx"
```

You can also run:

```
terraform output public_ip
```

and:

```
terraform output jenkins_url
```

Step 9 — Check EC2

Use the public IP:

```
ssh -i server-2.pem ec2-user@<PUBLIC-IP>
```

For example:

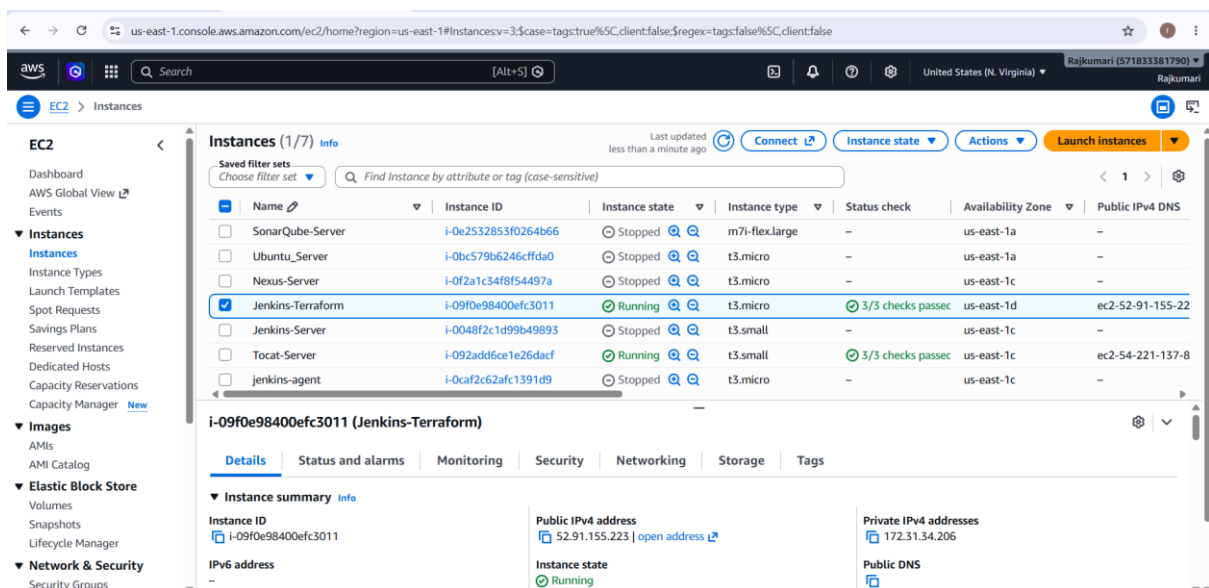
```
ssh -i server-2.pem ec2-user@54.xx.xx.xx
```

Once inside:

```
sudo systemctl status jenkins
```

You want:

Active: active (running)



The screenshot displays the AWS Management Console for the EC2 service. The left sidebar shows the navigation menu with 'Instances' selected. The main content area shows a list of instances. The 'Jenkins-Terraform' instance is selected and highlighted in blue. The instance is in a 'Running' state. The console shows a list of instances with columns for Name, Instance ID, Instance state, Instance type, Status check, Availability Zone, and Public IPv4 DNS. Below the list, the details for the 'Jenkins-Terraform' instance are shown, including its Instance ID, Public IPv4 address, Private IPv4 addresses, and Instance state.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
SonarQube-Server	i-0e2532853f0264b66	Stopped	m7i-flex.large	—	us-east-1a	—
Ubuntu_Server	i-0bc579b6246cfdad0	Stopped	t3.micro	—	us-east-1a	—
Nexus-Server	i-0f2a1c34f8f54497a	Stopped	t3.micro	—	us-east-1c	—
Jenkins-Terraform	i-09f0e98400efc3011	Running	t3.micro	3/3 checks passed	us-east-1d	ec2-52-91-155-22
Jenkins-Server	i-0048f2c1d99b49893	Stopped	t3.small	—	us-east-1c	—
Tocat-Server	i-092add6ce1e26dacf	Running	t3.small	3/3 checks passed	us-east-1c	ec2-54-221-137-8
jenkins-agent	i-0caf2c62afc1391d9	Stopped	t3.micro	—	us-east-1c	—

i-09f0e98400efc3011 (Jenkins-Terraform)

Instance summary

Instance ID: i-09f0e98400efc3011

Public IPv4 address: 52.91.155.223 | [open address](#)

Private IPv4 addresses: 172.31.34.206

Instance state: Running

Public DNS: [Public DNS](#)


```

root@ip-172-31-34-206:~
(2/2): java-21-amazon-corretto-headless-21.0.12+8-1.amzn2 66 MB/s | 96 MB 00:01
-----
Total 63 MB/s | 97 MB 00:01
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      : 1/1
  Installing     : java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_ 1/2
  Running scriptlet: java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_ 1/2
  Installing     : java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64 2/2
  Running scriptlet: java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64 2/2
  Verifying      : java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64 1/2
  Verifying      : java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_ 2/2

Installed:
  java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64
  java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64

complete!
[root@ip-172-31-34-206 ~]# java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[root@ip-172-31-34-206 ~]# sudo systemctl daemon-reload
sudo systemctl restart jenkins
[root@ip-172-31-34-206 ~]# sleep 5
[root@ip-172-31-34-206 ~]# sudo systemctl status jenkins --no-pager -l
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-08-27 14:58:45 UTC; 22s ago
     Main PID: 22167 (java)
       Tasks: 44 (limit: 1014)
      Memory: 327.4M
         CPU: 21.220s
    CGroup: /system.slice/jenkins.service
            └─22167 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.w
ar --webroot=/var/cache/jenkins/war --httpPort=8080

Aug 27 14:58:40 ip-172-31-34-206.ec2.internal jenkins[22167]: [LF]>
Aug 27 14:58:40 ip-172-31-34-206.ec2.internal jenkins[22167]: [LF]> *****
Aug 27 14:58:40 ip-172-31-34-206.ec2.internal jenkins[22167]: [LF]> *****
Aug 27 14:58:40 ip-172-31-34-206.ec2.internal jenkins[22167]: [LF]> *****
Aug 27 14:58:45 ip-172-31-34-206.ec2.internal jenkins[22167]: 2026-08-27 14:58:45.393+0000
[id=33] INFO jenkins.InitReactorRunner$1#onAttained: Completed initializati
on
Aug 27 14:58:45 ip-172-31-34-206.ec2.internal jenkins[22167]: 2026-08-27 14:58:45.412+0000
[id=24] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and ru
nning
Aug 27 14:58:45 ip-172-31-34-206.ec2.internal systemd[1]: Started jenkins.service - Jenkin
s Continuous Integration Server.
Aug 27 14:58:45 ip-172-31-34-206.ec2.internal jenkins[22167]: 2026-08-27 14:58:45.764+0000
[id=50] INFO h.m.DownloadService$Downloadable#load: Obtained the updated da
ta file for hudson.tasks.Maven.MavenInstaller
Aug 27 14:58:45 ip-172-31-34-206.ec2.internal jenkins[22167]: 2026-08-27 14:58:45.765+0000
[id=50] INFO hudson.util.Retrier#start: Performed the action check updates
server successfully at the attempt #1
Aug 27 14:58:50 ip-172-31-34-206.ec2.internal jenkins[22167]: 2026-08-27 14:58:50.729+0000
[id=68] WARNING h.n.DiskSpaceMonitorDescriptor#markNodeOfflineorOnline: Mak
ing Built-In Node offline temporarily due to the lack of disk space
[root@ip-172-31-34-206 ~]#

```

Step 12 — Get Jenkins Initial Password

Run:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy the output.

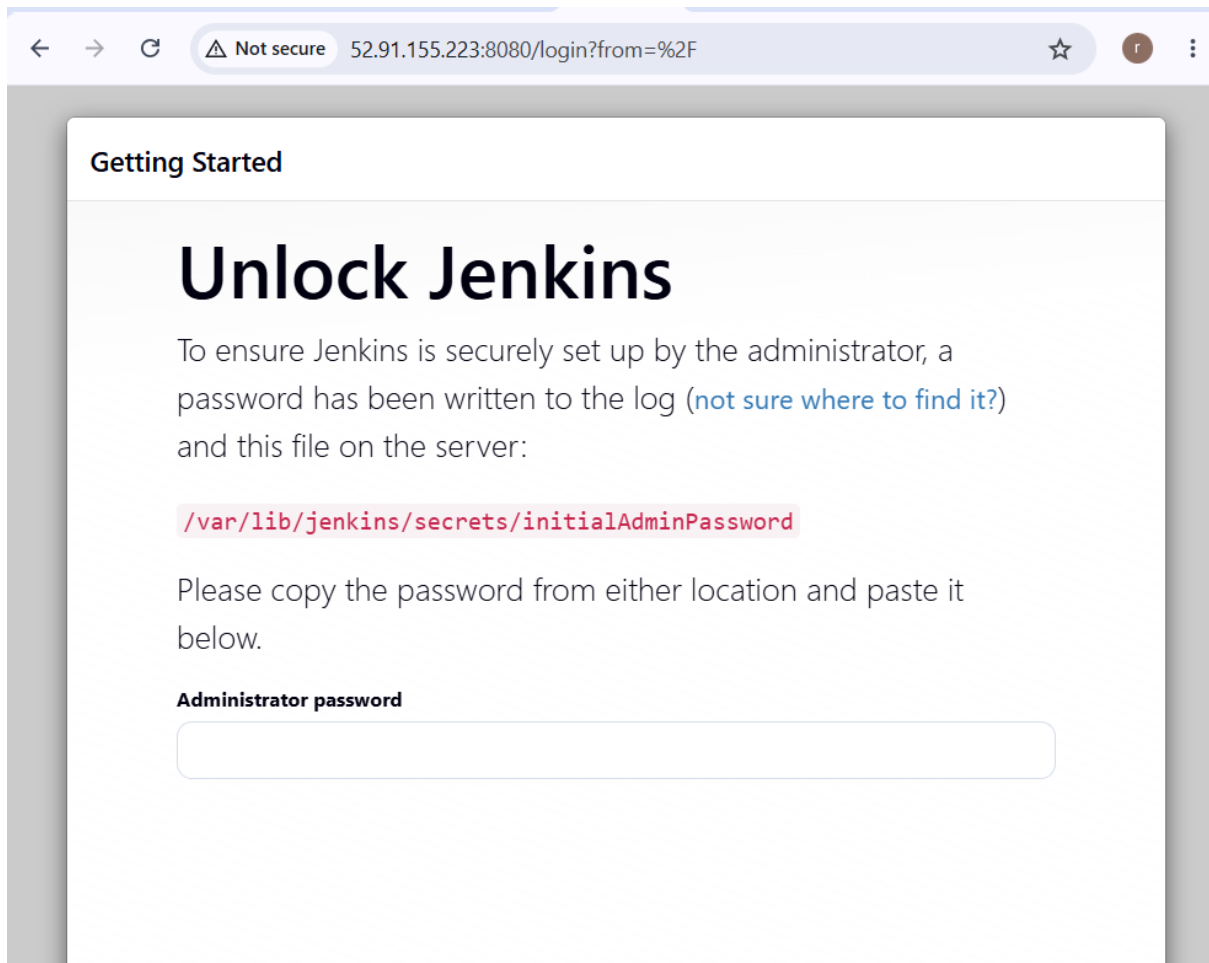
Then open this in your browser:

<http://<PUBLIC-IP>:8080>

For example:

<http://54.xx.xx.xx:8080>

Enter the initial password.



5. Create one Jenkins job to build and push the Docker image to Docker Hub.

Source: <https://github.com/betawins/Python-app.git>

Source Codes: <https://github.com/betawins/docker-tasks.git>

- From the **frontend source code**, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.
- From the **Java-based source code**, write a Dockerfile, build, run, and push the image to the Docker registry.
- From the **Node.js-based source code**, write a Dockerfile, build with tag v1, run, and push it to the Docker registry.
- Write a **docker-compose file** to set up WordPress with a MySQL database.

Overview

TASK 1: Jenkins Job → GitHub Python App → Docker Build → Docker Image → Docker Hub Push

TASK 2: Frontend Source → Dockerfile → Docker Build → Docker Run → Docker Hub

TASK 3: Java Source → Dockerfile → Docker Build → Docker Run → Docker Hub

TASK 4: Node.js Source → Dockerfile → Docker Build → Docker Image:v1 → Docker Run → Docker Hub

TASK 5: Docker Compose → WordPress + MySQL

Part 1 — Jenkins Job to Build and Push Docker Image

We will start with the Python application because you specifically provided:

Python-app GitHub Repository

The repository contains:

app.py

requirements.txt

Dockerfile

Jenkinsfile

according to the current repository listing

Step 1 — Verify Docker on Jenkins Server

SSH into your Jenkins EC2:

```
ssh -i server-2.pem ec2-user@<JENKINS-IP>
```

Check Docker:

```
docker --version
```

Example:

Docker version 28.x

Check Jenkins:

```
sudo systemctl status jenkins
```

```
root@ip-172-31-34-206:~  
Filesystem      Size  Used Avail Use% Mounted on  
tmpfs           2.0G  4.8M  2.0G   1% /tmp  
[root@ip-172-31-34-206 ~]# sudo systemctl restart jenkins  
[root@ip-172-31-34-206 ~]# sudo systemctl status jenkins  
● jenkins.service - Jenkins Continuous Integration Server  
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)  
   Active: active (running) since Thu 2026-08-27 15:21:33 UTC; 10s ago  
     Main PID: 40933 (java)  
       Tasks: 45 (limit: 1014)  
      Memory: 290.1M  
         CPU: 22.174s  
     CGroup: /system.slice/jenkins.service  
            └─40933 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.  
Aug 27 15:21:31 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:31.913+000>  
Aug 27 15:21:32 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:32.973+000>  
Aug 27 15:21:33 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:32.976+000>  
Aug 27 15:21:33 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:33.610+000>  
Aug 27 15:21:33 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:33.611+000>  
Aug 27 15:21:33 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:33.612+000>  
Aug 27 15:21:33 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:33.627+000>  
Aug 27 15:21:33 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:33.724+000>  
Aug 27 15:21:33 ip-172-31-34-206.ec2.internal jenkins[40933]: 2026-08-27 15:21:33.761+000>  
Aug 27 15:21:33 ip-172-31-34-206.ec2.internal systemd[1]: Started jenkins.service - Jenki>  
[root@ip-172-31-34-206 ~]# |
```

Step 2 — Add Jenkins User to Docker Group

This is important because Jenkins needs to execute Docker commands.

Run:

```
sudo usermod -aG docker jenkins
```

Then restart Jenkins:

```
sudo systemctl restart jenkins
```

Verify:

```
sudo -u jenkins docker ps
```

If you get:

permission denied

restart Jenkins again or restart the server:

```
sudo reboot
```

After reconnecting:

```
sudo -u jenkins docker ps
```

It should work.


```
root@ip-172-31-34-206:~  
[root@ip-172-31-34-206 ~]# sudo usermod -aG docker jenkins  
[root@ip-172-31-34-206 ~]# sudo systemctl restart jenkins  
[root@ip-172-31-34-206 ~]# sudo -u jenkins docker ps  
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES  
[root@ip-172-31-34-206 ~]# |
```

Step 3 — Create Docker Hub Repository

Go to:

[Docker Hub](#)

Create a repository such as:

python-app

Suppose your Docker Hub username is:

rajkumari

Your image name will be:

rajkumari/python-app:latest

[Repositories](#) / [Create](#)

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

Create repository

Repository Name *
python-app



Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. [Upgrade your plan to add more](#)

☒ **Public** 
Appears in Docker Hub search results

☐ **Private** 
Only visible to you

Cancel

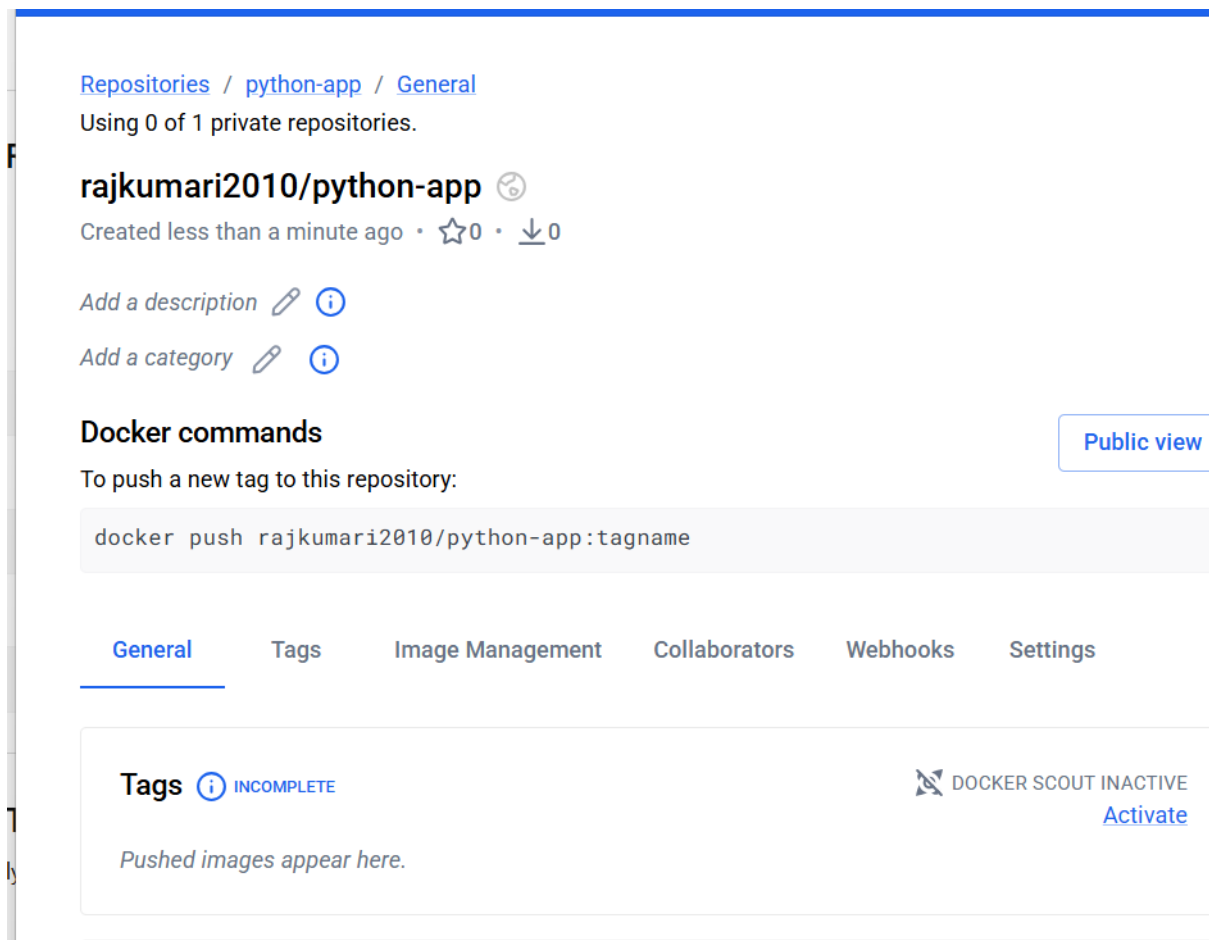
Create

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagname new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.



Step 4 — Create Docker Hub Token

Do **not** put your Docker Hub password directly in Jenkins.

Create a Docker Hub Personal Access Token.

In Docker Hub:

Account Settings



Personal access tokens



Generate new token

Copy the token.

You will use it as a Jenkins credential.



[Personal access tokens](#) / New access token

Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)

Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

Access token description
python-jenkins

Expires on
Never

Access permissions
Read & Write

To use the access token from your Docker CLI client:

1. Run

```
$ docker login -u rajkumari2010
```

Copy
2. At the password prompt, enter the personal access token.

```
dckr_pat_y31M_fDyN-ReMjqTF1nViVsCSCA
```

Copy

Back to access tokens

Step 5 — Add Docker Hub Credentials to Jenkins

Go to:

Jenkins



Manage Jenkins



Credentials



System



Global credentials



Add Credentials

Select:

Kind:

Username with password

Enter:

Username:

your Docker Hub username

Password:

Docker Hub PAT

For example:

Username: rajkumari

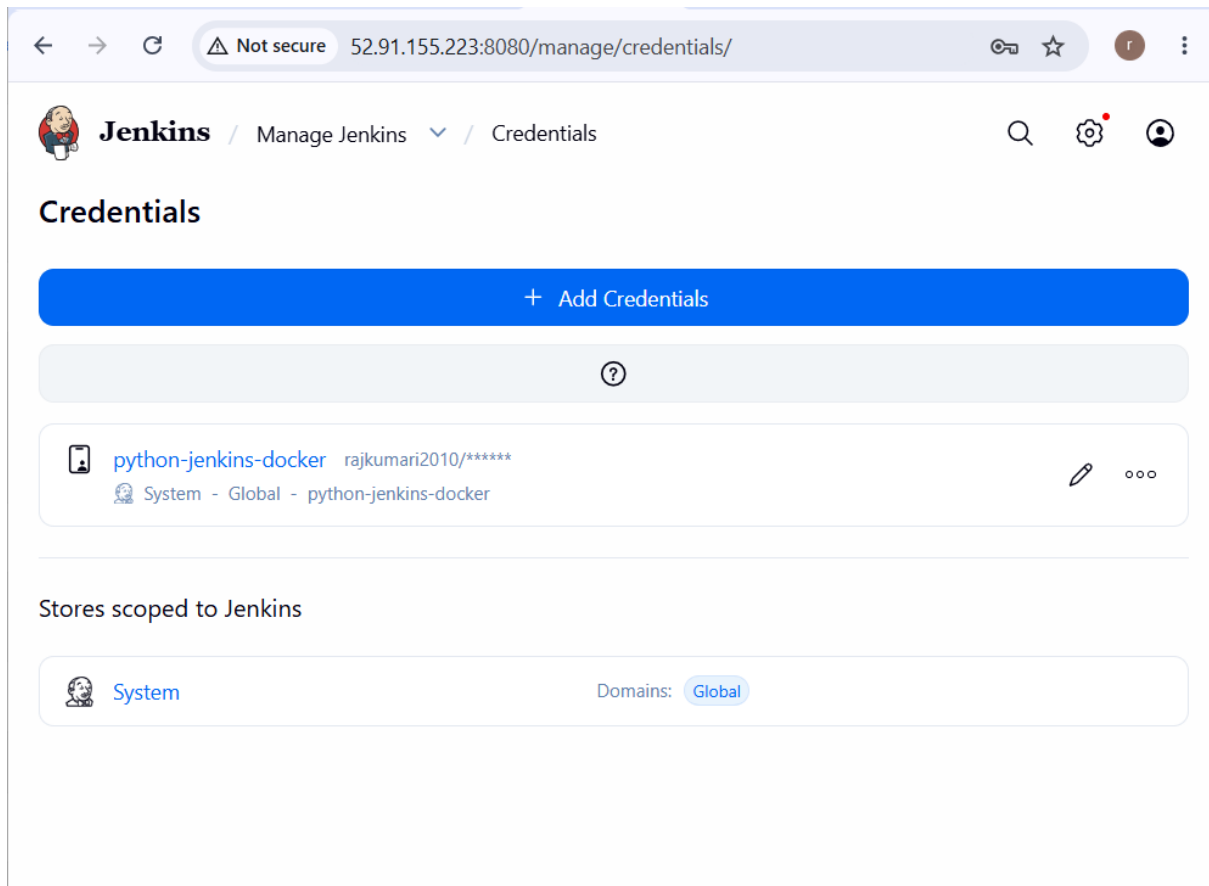
Password: *****

Credential ID:

dockerhub-credentials

Click:

Create



Step 6 — Create Jenkins Pipeline

Go to:

Jenkins Dashboard



New Item

Enter:

python-docker-build-push

Select:

Pipeline

Click:

OK

Step 7 — Add Pipeline

Go to:

Pipeline

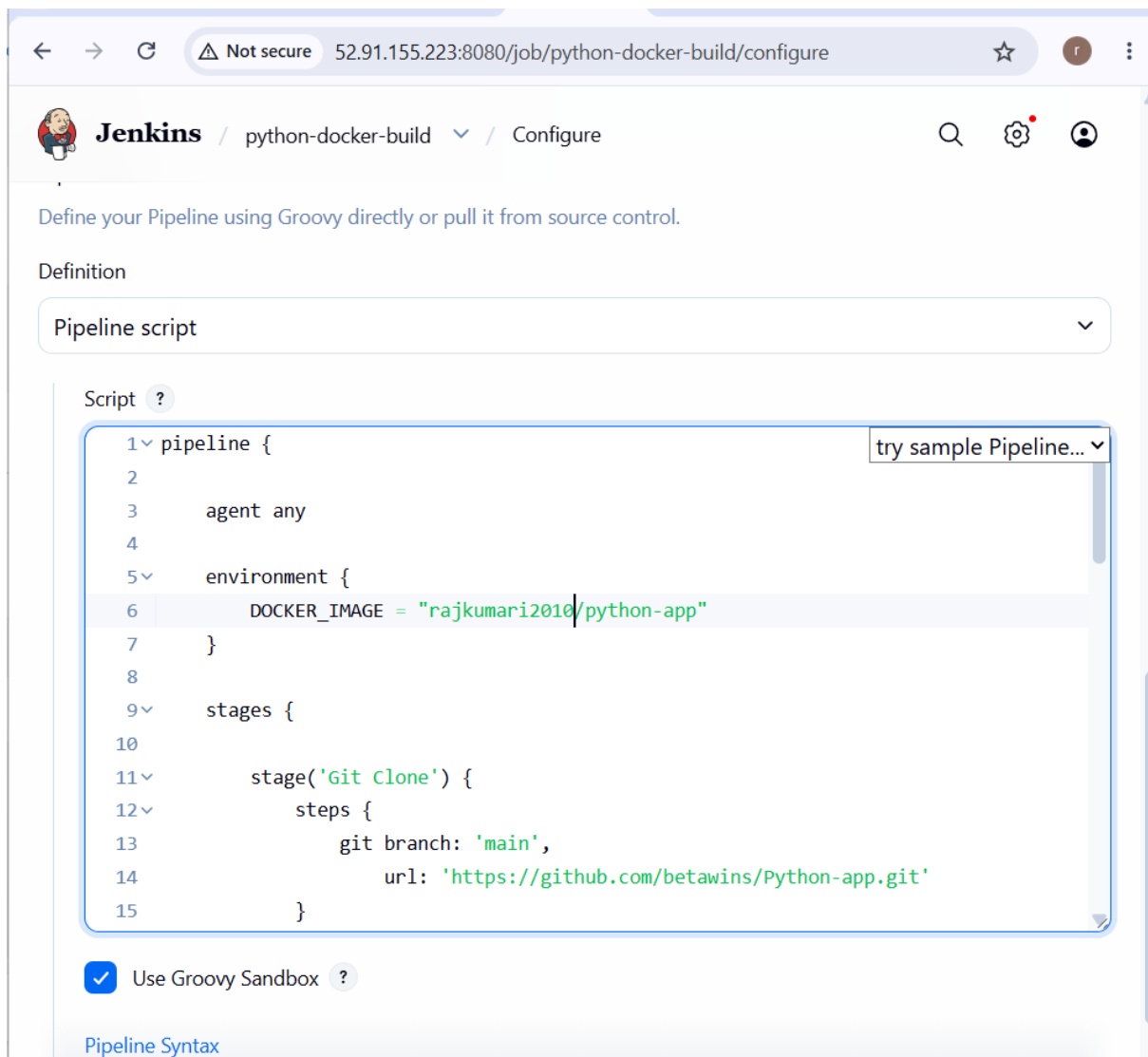
↓

Definition

↓

Pipeline script

Use this pipeline:



The screenshot shows the Jenkins web interface for configuring a pipeline. The browser address bar indicates the URL is `52.91.155.223:8080/job/python-docker-build/configure`. The Jenkins logo and job name 'python-docker-build' are visible in the header, along with a 'Configure' link. Below the header, a message states: 'Define your Pipeline using Groovy directly or pull it from source control.' Under the 'Definition' section, a dropdown menu is set to 'Pipeline script'. The main area is a text editor for the pipeline script, with a 'Script ?' label and a 'try sample Pipeline...' button. The script content is as follows:

```
1 pipeline {
2
3     agent any
4
5     environment {
6         DOCKER_IMAGE = "rajkumari2010/python-app"
7     }
8
9     stages {
10
11         stage('Git Clone') {
12             steps {
13                 git branch: 'main',
14                     url: 'https://github.com/betawins/Python-app.git'
15             }
16         }
17     }
18 }
```

At the bottom, there is a checkbox labeled 'Use Groovy Sandbox' which is checked, and a 'Pipeline Syntax' link.

Step 8 — Run Jenkins Job

Click:

Build Now

You should see:

Git Clone



Docker Build



Docker Login



Docker Push

The screenshot shows the Jenkins web interface for a pipeline named 'python-docker-build-push'. The browser address bar indicates the URL is '52.91.155.223:8080/job/python-docker-build-push/'. The Jenkins logo and the pipeline name are at the top. On the left, a sidebar contains links for 'Status', 'Changes', 'Build Now', 'Delete Pipeline', 'Configure', 'Rename', 'Stages', and 'Pipeline Syntax'. The main area shows a green checkmark icon next to the pipeline name. Below this, a 'Permalinks' section lists several links for different build numbers and times. On the left, a 'Builds' section shows a list of builds with a filter input and a dropdown menu. The builds listed are #6 at 3:53 PM and #5 at 3:52 PM, both with green status icons.

The screenshot shows the Jenkins web interface for the same pipeline, but now displaying the build log for build #5. The browser address bar shows the URL '52.91.155.223:8080/job/python-docker-build-push/5'. The Jenkins logo and the pipeline name are at the top. On the left, a sidebar contains links for 'Status', 'Changes', 'Build Now', 'Delete Pipeline', 'Configure', 'Rename', 'Stages', and 'Pipeline Syntax'. The main area shows the build log for build #5. The log content includes a list of Docker image digests, a 'latest' digest, and a message indicating that the Docker image was successfully pushed to Docker Hub. The log ends with 'Finished: SUCCESS'.

Step 9 — Verify Docker Image

On Jenkins:

docker images

You should see:

rajkumari/python-app latest

In Docker Hub:

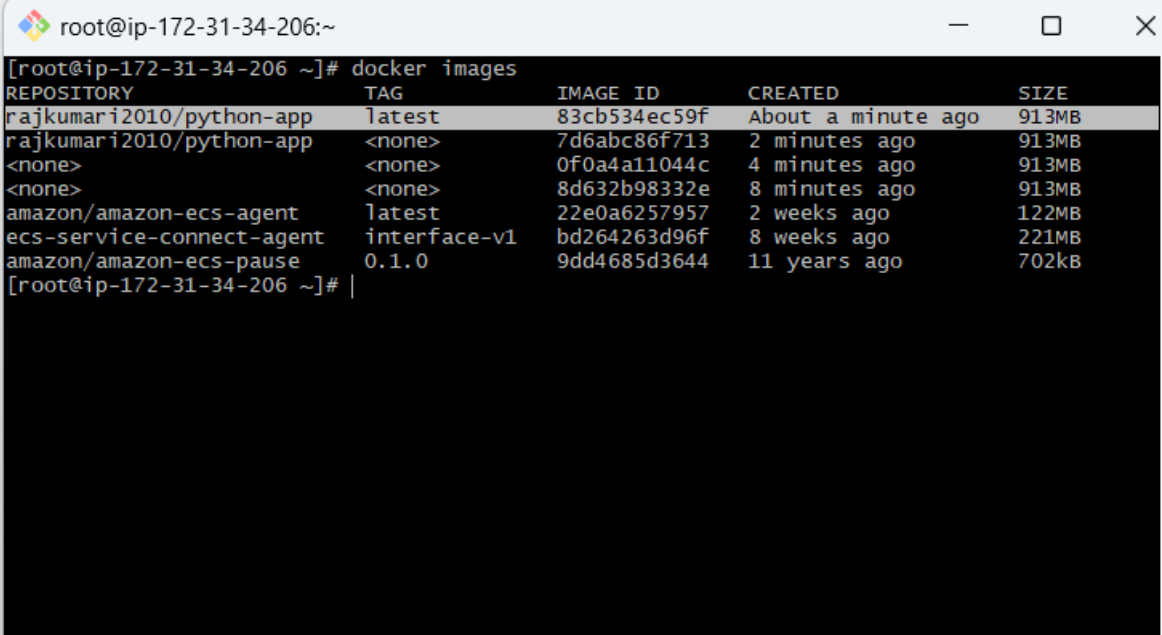
Repositories

↓

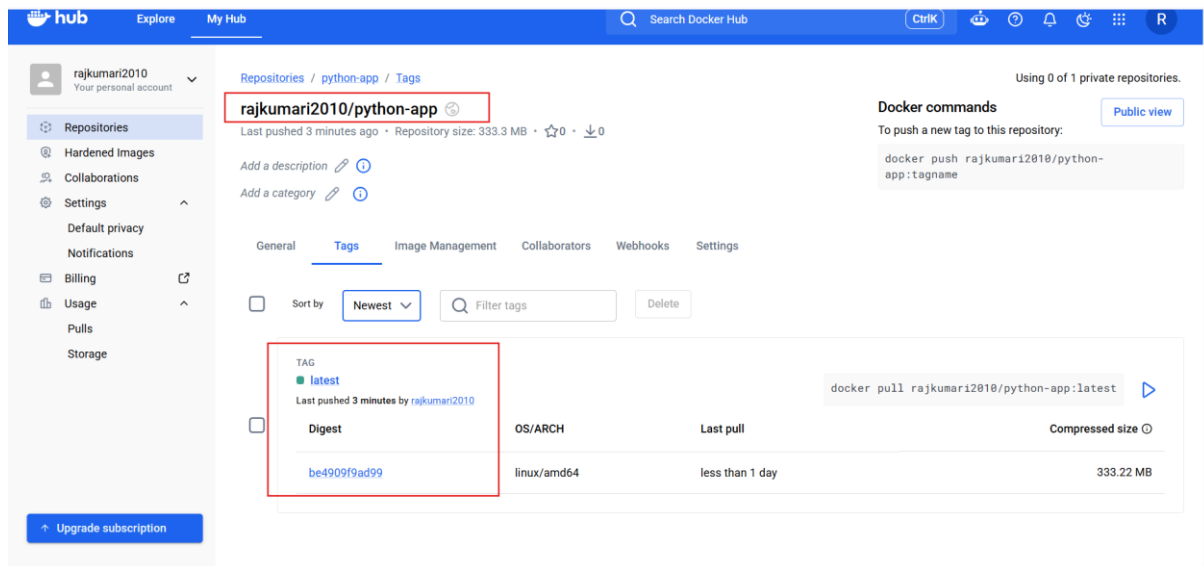
python-app

You should see:

latest

A terminal window titled 'root@ip-172-31-34-206:~' with standard window controls. The command 'docker images' has been executed, displaying a table of Docker images. The table has five columns: REPOSITORY, TAG, IMAGE ID, CREATED, and SIZE. The first row is highlighted in grey. The terminal output is as follows:

```
[root@ip-172-31-34-206 ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
rajkumari2010/python-app  latest             83cb534ec59f       About a minute ago  913MB
rajkumari2010/python-app  <none>             7d6abc86f713       2 minutes ago      913MB
<none>                 <none>             0f0a4a11044c       4 minutes ago      913MB
<none>                 <none>             8d632b98332e       8 minutes ago      913MB
amazon/amazon-ecs-agent  latest             22e0a6257957       2 weeks ago        122MB
ecs-service-connect-agent interface-v1        bd264263d96f       8 weeks ago        221MB
amazon/amazon-ecs-pause  0.1.0             9dd4685d3644       11 years ago       702kB
[root@ip-172-31-34-206 ~]# |
```



Part 2 — Frontend Source Code

Now complete the frontend assignment.

First clone the source repository:

```
git clone https://github.com/betawins/docker-tasks.git
```

Then:

```
cd docker-tasks
```

Check the files:

```
ls -la
```

Because the repository structure can change, **don't blindly create a Dockerfile in the repository root until you identify the frontend directory.**

Run:

```
find . -maxdepth 3 -type f | sort
```

Look for files such as:

```
package.json
```

```
index.html
```

```
src/
```

```
public/
```

or:

pom.xml

src/

or:

requirements.txt

This tells you which application you are working with.

Step 10 — Frontend Dockerfile

If the frontend is a React/Node frontend, a typical production Dockerfile is:

FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

FROM nginx:alpine

COPY --from=build /app/build /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

```
root@ip-172-31-34-206:~/docker-tasks/Frontend_based_source
FROM nginx:alpine
COPY . /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
~
~
~
~
~
~
```

Step 11 — Build Frontend Image

Example:

docker build -t my-frontend:v1 .

docker images

```
[root@ip-172-31-34-206 Frontend_based_source]# dokcer images
-bash: dokcer: command not found
[root@ip-172-31-34-206 Frontend_based_source]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my-frontend          v1                 65c01f68bf7b       About a minute ago 62.9MB
rajkumari2010/python-app   latest            83cb534ec59f       25 minutes ago    913MB
rajkumari2010/python-app   <none>            7d6abc86f713       26 minutes ago    913MB
<none>                 <none>            0f0a4a11044c       28 minutes ago    913MB
<none>                 <none>            8d632b98332e       32 minutes ago    913MB
amazon/amazon-ecs-agent   latest            22e0a6257957       2 weeks ago       122MB
ecs-service-connect-agent interface-v1       bd264263d96f       8 weeks ago       221MB
amazon/amazon-ecs-pause   0.1.0            9dd4685d3644       11 years ago      702kB
[root@ip-172-31-34-206 Frontend_based_source]#
```

Step 12 — Run Frontend Container

docker run -d \

--name frontend-container \

-p 8081:80 \

my-frontend:v1

Check:

docker ps

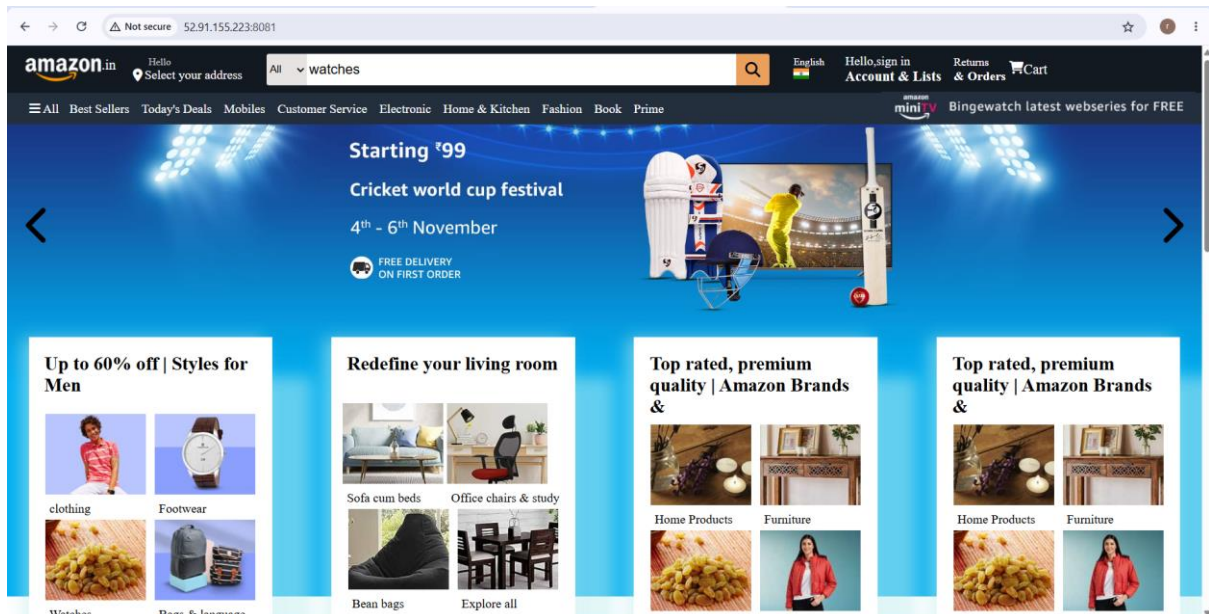
Test:

curl http://localhost:8081

Or open:

http://<SERVER-IP>:8081

Remember to allow TCP 8081 in the EC2 security group if you're accessing it externally.



Step 13 — Tag Frontend Image

Suppose your Docker Hub username is:

rajkumari

Run:

```
docker tag my-frontend:v1 \
```

```
rajkumari/my-frontend:v1
```

Step 14 — Login to Docker Hub

```
docker login
```

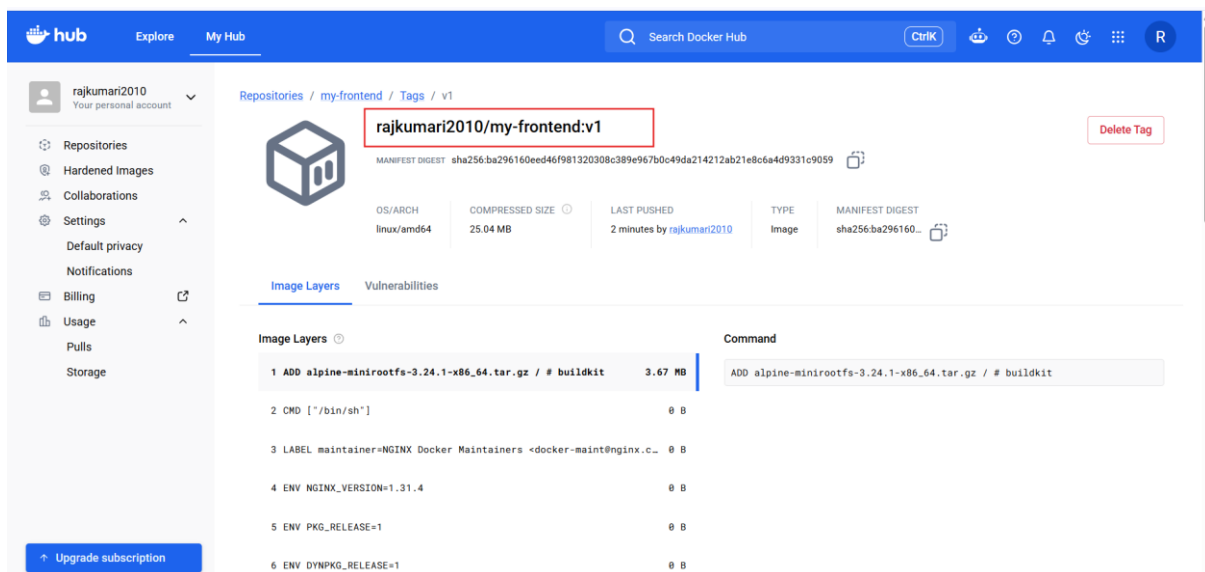
Enter your Docker Hub username and PAT.

Step 15 — Push Frontend Image

`docker push rajkumari/my-frontend:v1`

Verify in Docker Hub.

```
root@ip-172-31-34-206:~/docker-tasks/Frontend_based_source
[root@ip-172-31-34-206 Frontend_based_source]# docker push rajkumari2010/my-frontend:v1
The push refers to repository [docker.io/rajkumari2010/my-frontend]
2d06bc42db26: Layer already exists
edc23e36f952: Layer already exists
628c86fdf4c1: Layer already exists
63c800442e2a: Layer already exists
31e777702b00: Layer already exists
e3b86476d58e: Layer already exists
18ca587a5d44: Layer already exists
1c4d71cf6bfe: Layer already exists
34884abbe928: Layer already exists
v1: digest: sha256:ba296160eed46f981320308c389e967b0c49da214212ab21e8c6a4d9331c9059 size: 2197
[root@ip-172-31-34-206 Frontend_based_source]# |
```



Part 3 — Java Application

Now identify the Java source code in docker-tasks.

Look for:

`pom.xml`

or:

build.gradle

Step 16 — Build Java Image

Go into the Java application directory:

```
cd <java-directory>
```

Build:

```
docker build -t my-java-app:v1 .
```

Check:

docker images

```
root@ip-172-31-30-131:~/docker-tasks/Java_based_source
=> => extracting sha256:e703c6a2a25b813beb3663ee45d1066d788e7f568a22e9daa27d1efe2357217c 0.0s
=> => extracting sha256:40b806e03ebadbfe6ea2310fb23f6be5154ff8709e747a0043079de0b862d28c 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 1.89MB 0.1s
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:17-jre@sha256:13cc28a6cc72a38ce1f00c906be3 5.6s
=> => resolve docker.io/library/eclipse-temurin:17-jre@sha256:13cc28a6cc72a38ce1f00c906be3580c1a3e 0.0s
=> => sha256:c984fca94ab42452795a0bee45a35a92318390a673d890b8216e9677ac21aa37 2.13kB / 2.13kB 0.0s
=> => sha256:13cc28a6cc72a38ce1f00c906be3580c1a3e604b8984d694f369a96742abc93b 7.22kB / 7.22kB 0.0s
=> => sha256:9a3b1de629c51deedfee94b5b33721d99e4b1f0f208bead2879ec457b006cbea 8.99kB / 8.99kB 0.0s
=> => sha256:409c221e19629281dd0400761b763523ed5171be79fd0c00b7401bfc52326b16 47.52MB / 47.52MB 1.8s
=> => sha256:1a085c661b8f2d3ba3e791ae4405a9dd67c4be765d803635878ce9d1eea51e60 20.13MB / 20.13MB 1.6s
=> => extracting sha256:1a085c661b8f2d3ba3e791ae4405a9dd67c4be765d803635878ce9d1eea51e60 1.3s
=> => sha256:d12223f877f052485212a69a1db48316ea7dd9bfe606c4a6cf8d9d92dbba53e5 158B / 158B 1.7s
=> => sha256:074b156e16ddb041640e80a64efb5d1c599275c1b0a98c84d95a5fc3ac83951b 2.46kB / 2.46kB 1.8s
=> => extracting sha256:409c221e19629281dd0400761b763523ed5171be79fd0c00b7401bfc52326b16 2.2s
=> => extracting sha256:d12223f877f052485212a69a1db48316ea7dd9bfe606c4a6cf8d9d92dbba53e5 0.0s
=> => extracting sha256:074b156e16ddb041640e80a64efb5d1c599275c1b0a98c84d95a5fc3ac83951b 0.0s
=> [stage-1 2/3] WORKDIR /app 1.2s
=> [build 2/5] WORKDIR /app 0.2s
=> [build 3/5] COPY pom.xml . 0.1s
=> [build 4/5] COPY src ./src 0.1s
=> [build 5/5] RUN mvn clean package -DskipTests 24.4s
=> [stage-1 3/3] COPY --from=build /app/target/*.jar app.jar 0.1s
=> exporting to image 0.1s
=> => exporting layers 0.1s
=> => writing image sha256:598adabfdfaec06456b9afd0804be1b134abc6c85776ce583cf2529d2003c318 0.0s
=> => naming to docker.io/library/my-java-app:v1 0.0s
root@ip-172-31-30-131 Java_based_source]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED
my-java-app	v1	598adabfdfae	About a minute ago
571833381790.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx	v1	22b7519cba94	6 hours ago
my-custom-nginx	v1	22b7519cba94	6 hours ago
rajkumari2010/my-custom-nginx	v1	22b7519cba94	6 hours ago
jenkins-docker-demo	v1	04f7a31a23ca	10 hours ago
rajkumari2010/jenkins-docker-demo	1	04f7a31a23ca	10 hours ago
rajkumari2010/jenkins-docker-demo	2	04f7a31a23ca	10 hours ago
rajkumari2010/jenkins-docker-demo	3	04f7a31a23ca	10 hours ago
rajkumari2010/jenkins-docker-demo	4	04f7a31a23ca	10 hours ago
rajkumari2010/jenkins-docker-demo	5	04f7a31a23ca	10 hours ago

Step 17 — Run Java Container

```
docker run -d \  
  --name java-container \  
  -p 8082:8080 \  
  my-java-app:v1
```

Check:

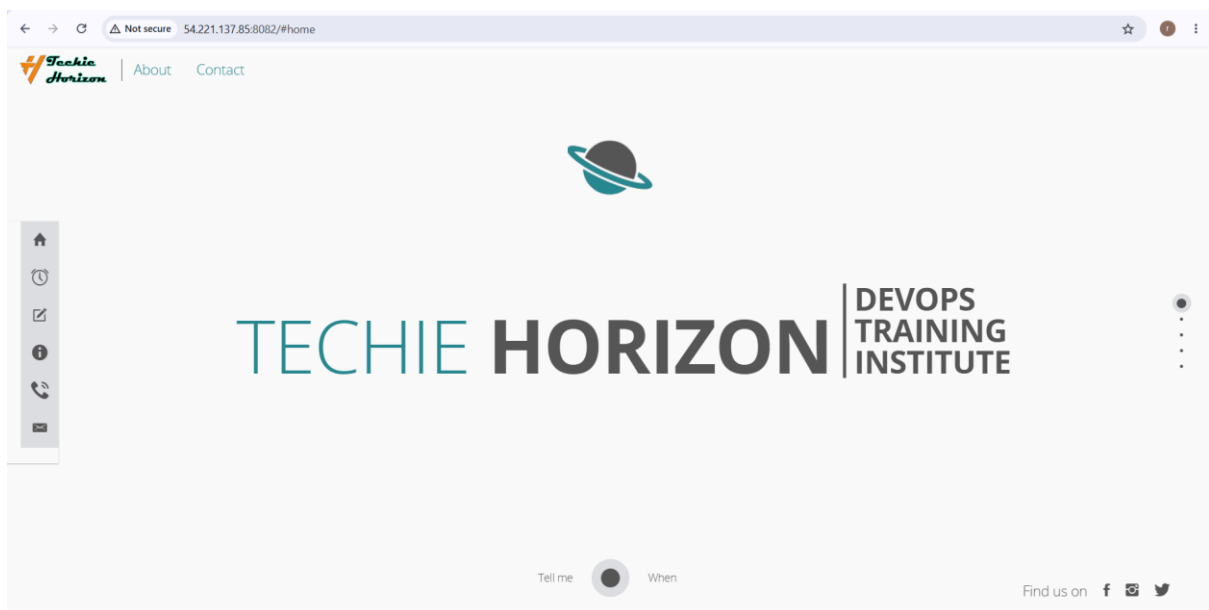
```
docker ps
```

Check logs:

```
docker logs java-container
```

Test:

```
curl http://localhost:8082
```



tep 18 — Tag Java Image

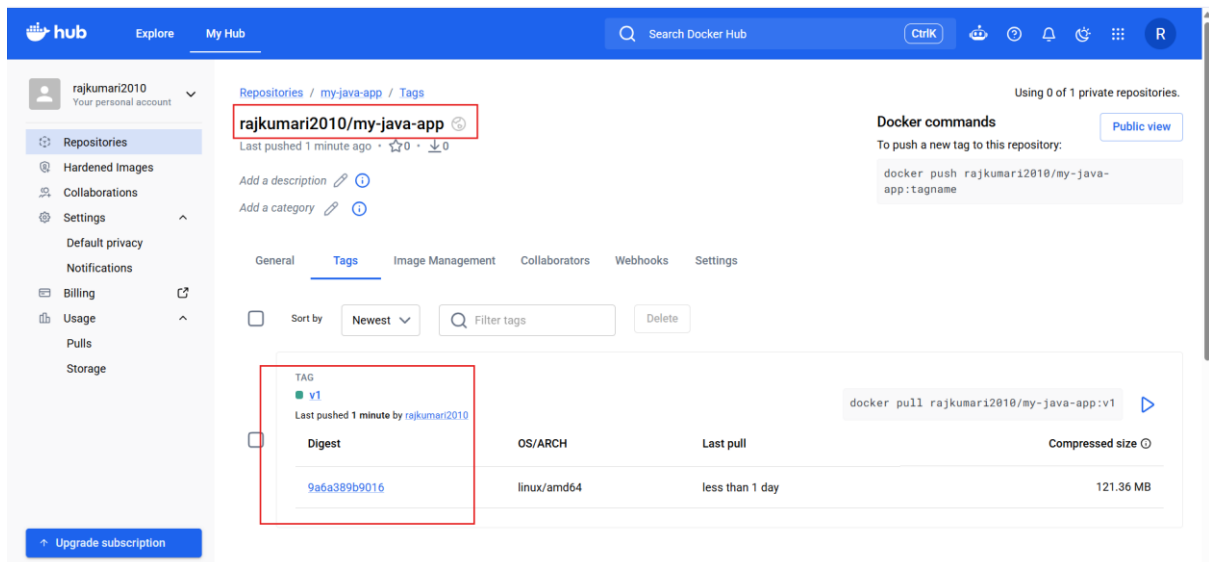
```
docker tag my-java-app:v1 \  
rajkumari/my-java-app:v1
```

Push:

```
docker push rajkumari/my-java-app:v1
```



```
root@ip-172-31-30-131:~/docker-tasks/Java_based_source
root@ip-172-31-30-131 Java_based_source]# docker run -d \
--name java-container \
-p 8082:8080 \
my-java-app:v1
09986609848b68b7dfc4493fc3b692b6d2b797633f57f81ac9c08118
root@ip-172-31-30-131 Java_based_source]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
09986609848b   my-java-app:v1                     "/__cacert_entrypoin..." 11 seconds ago Up 10 seconds  0.0.0.0:8082->8080/tcp, :::8082->8080/tcp
root@ip-172-31-30-131 Java_based_source]# docker tag my-java-app:v1 \
rajkumari2010/my-java-app:v1
root@ip-172-31-30-131 Java_based_source]# docker push rajkumari2010/my-java-app:v1
The push refers to repository [docker.io/rajkumari2010/my-java-app]
5c913a3396e1: Pushed
ec593f7527aa: Pushed
281852bcff0e: Mounted from library/eclipse-temurin
1d4ad11192b0: Mounted from library/eclipse-temurin
d1cb3b727c6b: Mounted from library/eclipse-temurin
9c9c5161ed13: Mounted from library/eclipse-temurin
168f4b64baaa: Mounted from library/eclipse-temurin
b577adbc0b58: Mounted from library/eclipse-temurin
v1: digest: sha256:9a6a389b9016fb41fe757b19e33f2df64f0042a10a99329f715666df789c533b size: 1993
root@ip-172-31-30-131 Java_based_source]#
```



Part 4 — Node.js Application

Find the Node application:

find . -name package.json

Go into its directory:

cd <node-directory>

```
root@ip-172-31-30-131:~/docker-tasks/NodeJs_based_source
[root@ip-172-31-30-131 docker-tasks]# ll
total 4
drwxr-xr-x. 2 root root 82 Aug 27 17:28 Frontend_based_source
drwxr-xr-x. 3 root root 50 Aug 27 17:30 Java_based_source
drwxr-xr-x. 4 root root 76 Aug 27 17:28 NodeJs_based_source
-rw-r--r--. 1 root root 488 Aug 27 17:28 README.md
[root@ip-172-31-30-131 docker-tasks]# cd NodeJs_based_source/
[root@ip-172-31-30-131 NodeJs_based_source]# ll
-bash: ll: command not found
[root@ip-172-31-30-131 NodeJs_based_source]# ll
total 1212
-rw-r--r--. 1 root root 1234987 Aug 27 17:28 package-lock.json
-rw-r--r--. 1 root root 977 Aug 27 17:28 package.json
drwxr-xr-x. 2 root root 120 Aug 27 17:28 public
drwxr-xr-x. 4 root root 99 Aug 27 17:28 src
[root@ip-172-31-30-131 NodeJs_based_source]# vi Dockerfile
[root@ip-172-31-30-131 NodeJs_based_source]# cat package.json
{
  "name": "zomato_clone",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@emotion/react": "^11.10.5",
    "@emotion/styled": "^11.10.5",
    "@mui/icons-material": "^5.11.0",
    "@mui/material": "^5.11.1",
    "@testing-library/jest-dom": "^5.16.5",
    "@testing-library/react": "^13.4.0",
    "@testing-library/user-event": "^13.5.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "sass": "^1.57.1",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "eslintConfig": {
    "extends": [
      "react-app",
      "react-app/jest"
    ]
  }
}
```

Vi Dockerfile

```
root@ip-172-31-30-131:~/docker-tasks/NodeJs_based_source
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Step 19 — Build Node Image With v1

Your assignment specifically says:

Build with tag v1

Run:

```
docker build -t my-node-app:v1 .
```

Verify:

```
docker images
```

You should see:

```
my-node-app  v1
```

```
root@ip-172-31-30-131:~/docker-tasks/NodeJs_based_source
[root@ip-172-31-30-131 NodeJs_based_source]# docker build -t my-node-app:v1 .
[+] Building 80.0s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 216B                                0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine  0.4s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632 2.5s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632 0.0s
=> => sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 7.67kB / 7.67kB 0.0s
=> => sha256:afdf98210b07b586eb71fa22ba2e432e058e4cd1304d31ed60888755b8c865fb 1.72kB / 1.72kB 0.0s
=> => sha256:11cedc39e663e7c5d5cb9cc77a461a0d2adc25537b94e6831a6108f09cb2001b 6.52kB / 6.52kB 0.0s
=> => sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 43.23MB / 43.23MB 0.5s
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 1.26MB / 1.26MB 0.1s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.2s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 0.1s
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 445B / 445B 0.2s
=> => extracting sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 1.6s
=> => extracting sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 0.1s
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 0.0s
=> [internal] load build context                                   0.1s
=> => transferring context: 4.13MB                                   0.1s
=> [2/5] WORKDIR /app                                             0.3s
=> [3/5] COPY package*.json ./                                    0.0s
=> [4/5] RUN npm install                                          41.3s
=> [5/5] COPY . .                                                 0.2s
=> exporting to image                                             35.1s
=> => exporting layers                                             35.0s
=> => writing image sha256:174758d20dbf991e8ce90a2db3a5689817567dcb14a9e9c9e18536d840264a48 0.0s
=> => naming to docker.io/library/my-node-app:v1                  0.0s
[root@ip-172-31-30-131 NodeJs_based_source]#
```

Step 20 — Run Node Application

If the application listens on port 3000:

```
docker run -d \
```

```
--name node-container \
```

```
-p 3000:3000 \
```

```
my-node-app:v1
```

Check:

```
docker ps
```

Logs:

```
docker logs node-container
```

Test:

```
curl http://localhost:3000
```



Step 21 — Tag Node Image for Docker Hub

```
docker tag my-node-app:v1 \
rajkumari/my-node-app:v1
```

Step 22 — Push Node Image

```
docker push rajkumari/my-node-app:v1
```

Now Docker Hub should contain:

rajkumari/

|

└─ python-app:latest

└─ my-frontend:v1

└─ my-java-app:v1

└─ my-node-app:v1

```
root@ip-172-31-30-131:~/docker-tasks/NodeJs_based_source
[root@ip-172-31-30-131 NodeJs_based_source]# docker run -d \
--name node-container \
-p 3000:3000 \
my-node-app:v1
d194d67b66760c7928af72ef95b73d099446f131436c9958783252e757a7f2a2
[root@ip-172-31-30-131 NodeJs_based_source]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
d194d67b6676   my-node-app:v1 "docker-entrypoint.s..." 8 seconds ago  Up 7 seconds  0.0.0.0:3000->3000/tcp
09986609848b   my-java-app:v1 "/_cacert_entrypoin..." 10 minutes ago Up 10 minutes  0.0.0.0:8082->8080/tcp
[root@ip-172-31-30-131 NodeJs_based_source]# docker tag my-node-app:v1 \
rajkumari/my-node-app:v1
[root@ip-172-31-30-131 NodeJs_based_source]# docker tag my-node-app:v1 rajkumari2010/my-node-app:v1
[root@ip-172-31-30-131 NodeJs_based_source]# docker push rajkumari2010/my-node-app:v1
The push refers to repository [docker.io/rajkumari2010/my-node-app]
e7bbf23936e8: Pushed
b69872f033fc: Pushed
b0e129a65914: Pushed
41bc30b2a377: Pushed
afa543f85b46: Mounted from library/node
e10358715ead: Mounted from library/node
4983b93ee796: Mounted from library/node
29df493baa13: Mounted from library/node
v1: digest: sha256:b0dedad46e6950e7e5a721ea6b98af5289ec40c70da0b3b2393f2e357c8810e9 size: 1998
[root@ip-172-31-30-131 NodeJs_based_source]#
```

rajkumari2010
Your personal account

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Upgrade subscription

Repositories / my-node-app / General

rajkumari2010/my-node-app

Last pushed 2 minutes ago · ☆0 · ↓0

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
v1		Image	less than 1 day	2 minutes

See all

Repository overview

An overview describes what your image does and how to run it. It displays in the public view of your

Using 0 of 1 private repositories.

Docker commands

To push a new tag to this repository:

docker push rajkumari2010/my-node-app:tagname

Public view

buildcloud

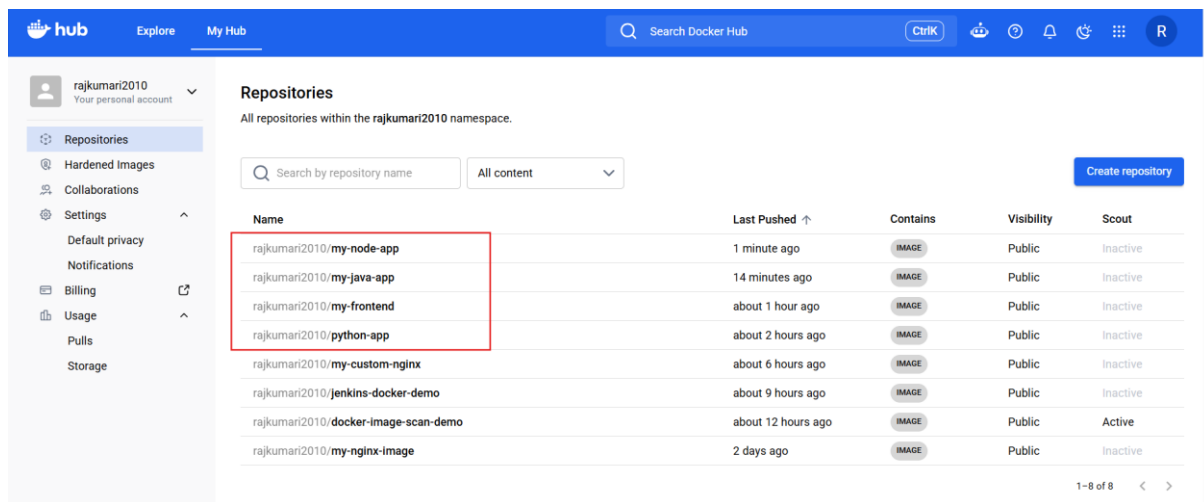
Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

Go to Docker Build Cloud



Part 5 — WordPress + MySQL Using Docker Compose

Now create a separate directory:

```
mkdir wordpress-compose
```

```
cd wordpress-compose
```

vi compose.yaml

```
root@ip-172-31-30-131:~/wordpress-compose
services:
  db:
    image: mysql:8.0
    container_name: wordpress-db

    restart: unless-stopped

    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress_password
      MYSQL_ROOT_PASSWORD: root_password

    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress

    restart: unless-stopped

    ports:
      - "8085:80"

    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress_password
      WORDPRESS_DB_NAME: wordpress

    depends_on:
      - db

    volumes:
      - wordpress_data:/var/www/html

volumes:
  db_data:
  wordpress_data:
```

Step 24 — Start WordPress

Run:

`docker compose up -d`

You should see:

[+] Running 2/2

✓ Container wordpress-db

✓ Container wordpress

Step 25 — Check Containers

`docker compose ps`

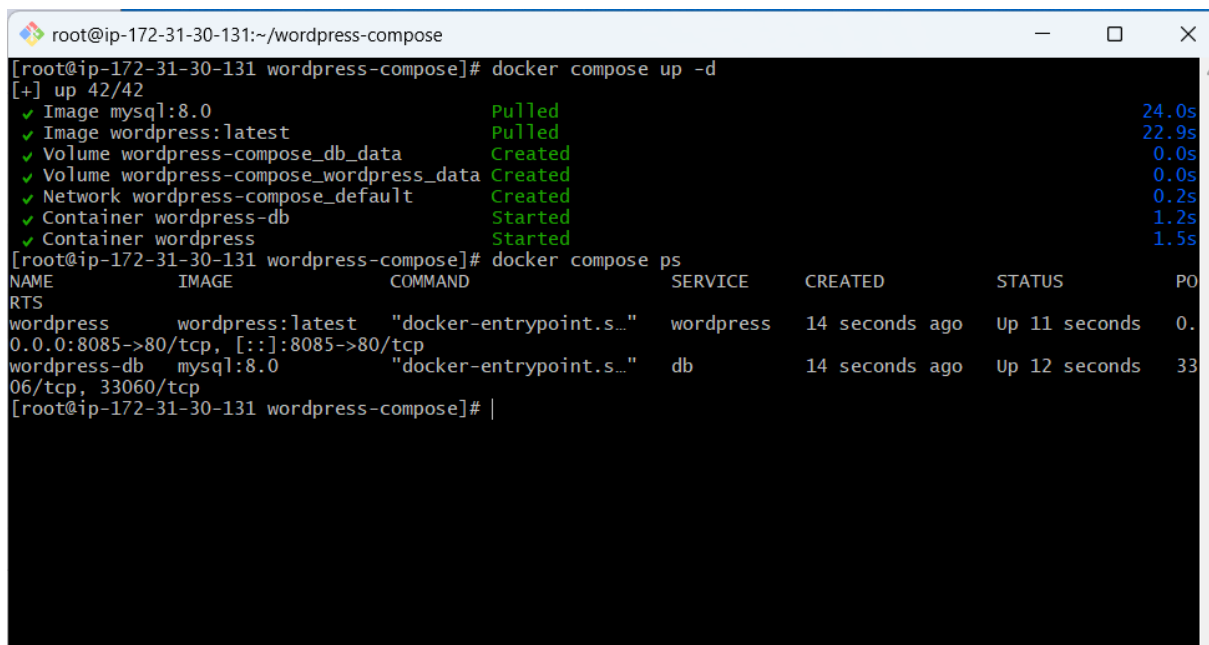
You should have:

wordpress-db

wordpress

Or:

`docker ps`

A terminal window titled 'root@ip-172-31-30-131:~/wordpress-compose' showing the execution of 'docker compose up -d' and 'docker compose ps'. The first command shows the status of various components: mysql:8.0 (Pulled, 24.0s), wordpress:latest (Pulled, 22.9s), and several volumes and networks (Created, 0.0s to 0.2s). The 'wordpress-db' and 'wordpress' containers are shown as 'Started' (1.2s and 1.5s respectively). The second command, 'docker compose ps', displays a table of the running containers.

```
root@ip-172-31-30-131:~/wordpress-compose
[root@ip-172-31-30-131 wordpress-compose]# docker compose up -d
[+] up 42/42
✓ Image mysql:8.0 Pulled 24.0s
✓ Image wordpress:latest Pulled 22.9s
✓ Volume wordpress-compose_db_data Created 0.0s
✓ Volume wordpress-compose_wordpress_data Created 0.0s
✓ Network wordpress-compose_default Created 0.2s
✓ Container wordpress-db Started 1.2s
✓ Container wordpress Started 1.5s
[root@ip-172-31-30-131 wordpress-compose]# docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
wordpress wordpress:latest "docker-entrypoint.s..." wordpress 14 seconds ago Up 11 seconds 0.0.0.0:8085->80/tcp, [::]:8085->80/tcp
wordpress-db mysql:8.0 "docker-entrypoint.s..." db 14 seconds ago Up 12 seconds 3306/tcp, 33060/tcp
[root@ip-172-31-30-131 wordpress-compose]# |
```

Step 26 — Check Logs

WordPress:

`docker compose logs wordpress`

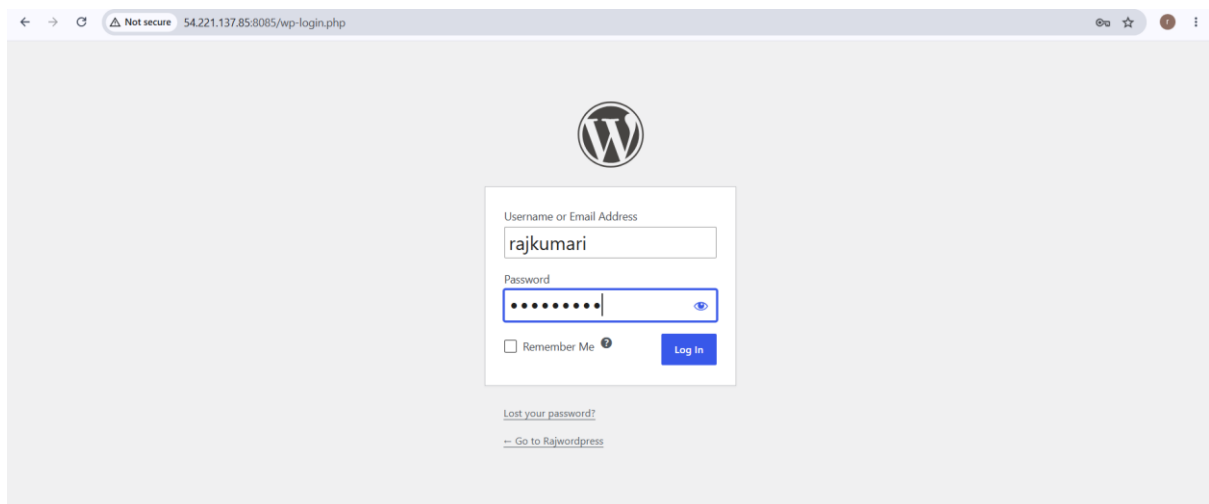
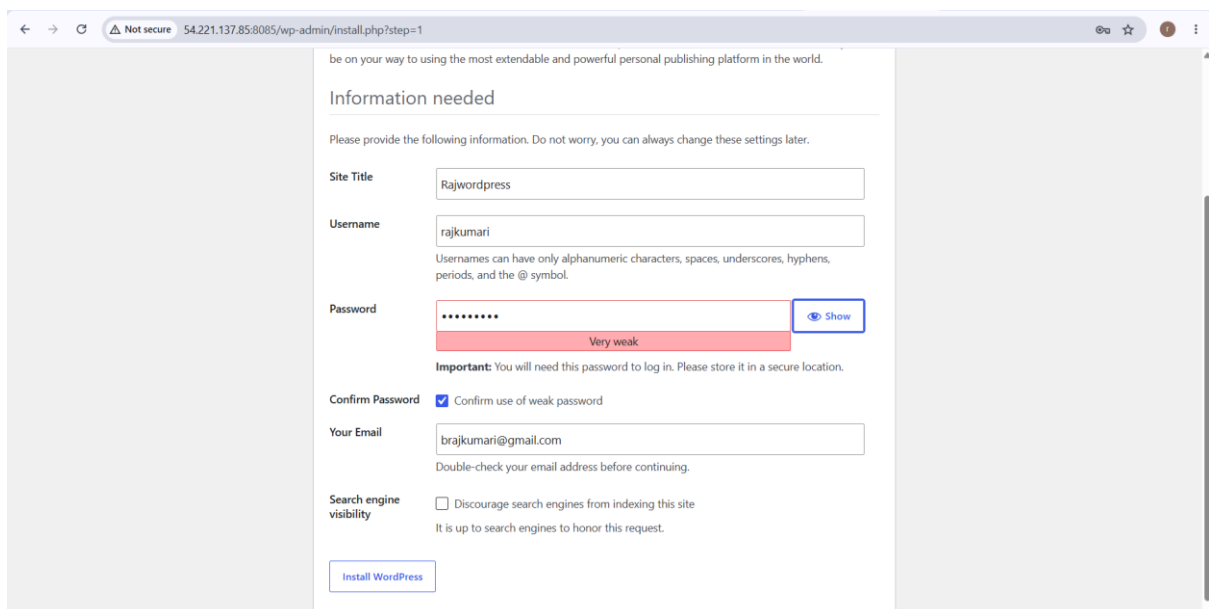
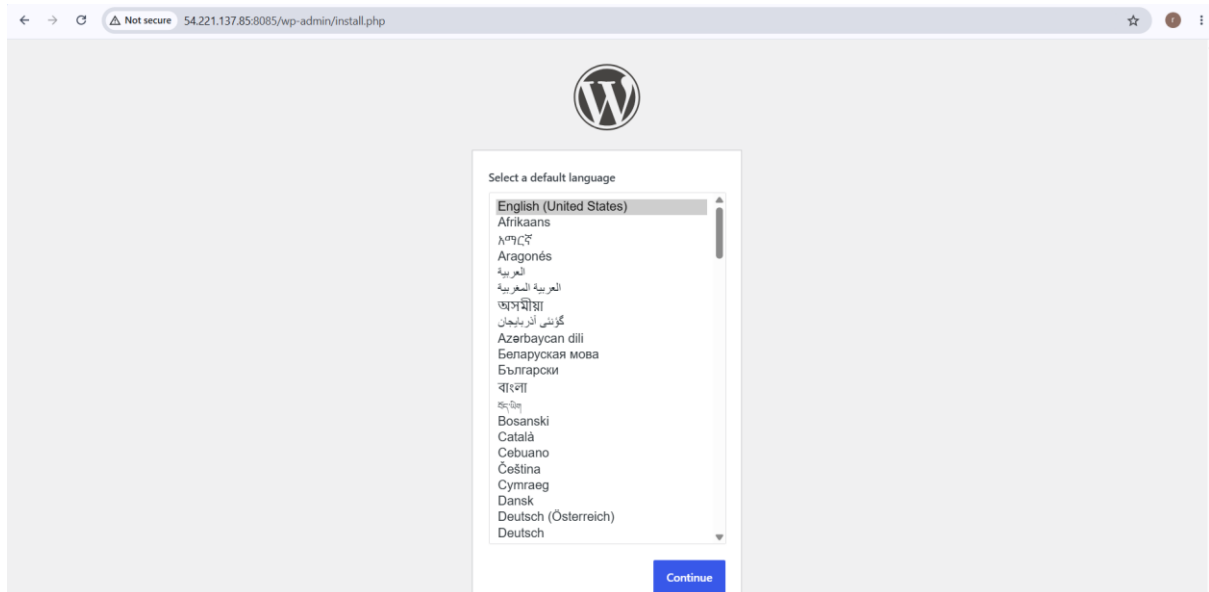
MySQL:

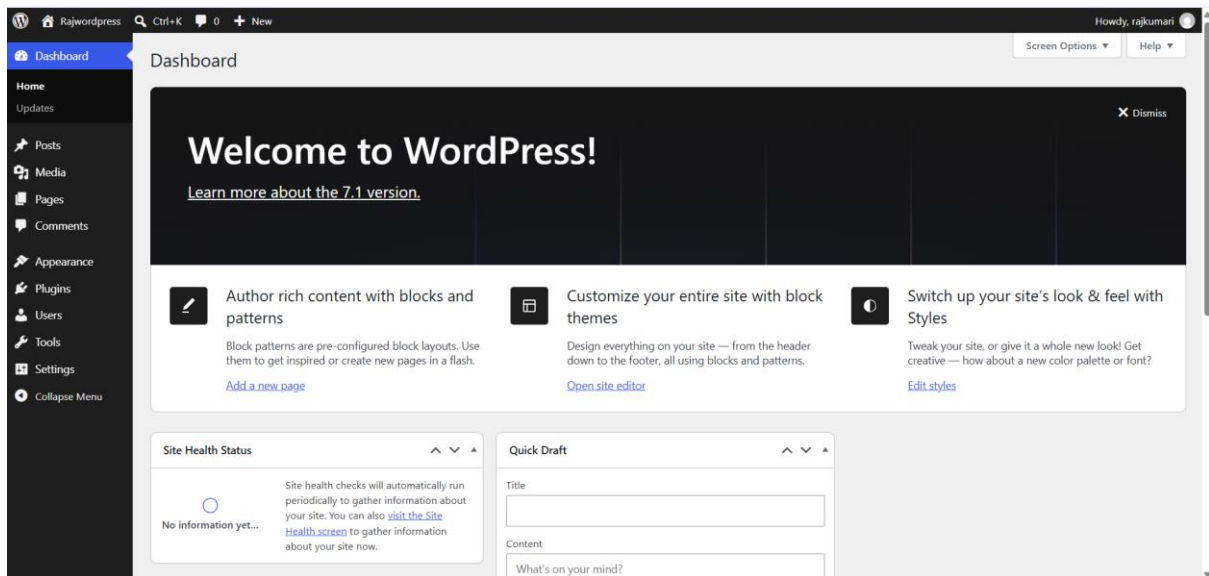
`docker compose logs db`

If MySQL is still initializing, wait a little and check again.

Step 27 — Open WordPress

Open: <http://<SERVER-IP>:8085>





Step 28 — Verify Docker Compose Networking

Notice this:

WORDPRESS_DB_HOST: db:3306

```

root@ip-172-31-30-131:~/wordpress-compose
[root@ip-172-31-30-131 wordpress-compose]# docker compose up -d
[+] up 2/2
  Container wordpress      Running
  Container wordpress-db   Running
[root@ip-172-31-30-131 wordpress-compose]# docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PO
wordpress           wordpress:latest     "docker-entrypoint.s..." wordpress  14 minutes ago Up 14 minutes  0.
0.0.0:8085->80/tcp, [::]:8085->80/tcp
wordpress-db        mysql:8.0            "docker-entrypoint.s..." db         14 minutes ago Up 14 minutes  33
0.0.0:3306->3306/tcp
[root@ip-172-31-30-131 wordpress-compose]# docker compose logs db
2026-08-27 18:02:17+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1
.el9 started.
2026-08-27 18:02:18+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-08-27 18:02:18+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1
.el9 started.
2026-08-27 18:02:18+00:00 [Note] [Entrypoint]: Initializing database files
2026-08-27T18:02:18.764749Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache
' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
wordpress-db | 2026-08-27T18:02:18.765694Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.0.4
6) initializing
wordpress-db | 2026-08-27T18:02:18.788000Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has star
ted.
wordpress-db | 2026-08-27T18:02:19.489828Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ende
d.
wordpress-db | 2026-08-27T18:02:21.108595Z 6 [Warning] [MY-010453] [Server] root@localhost is created wit
h an empty password! Please consider switching off the --initialize-insecure option.
wordpress-db | 2026-08-27 18:02:24+00:00 [Note] [Entrypoint]: Database files initialized
wordpress-db | 2026-08-27 18:02:24+00:00 [Note] [Entrypoint]: Starting temporary server
wordpress-db | 2026-08-27T18:02:25.342487Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache
' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
wordpress-db | 2026-08-27T18:02:25.345191Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.4
6) starting as process 121
wordpress-db | 2026-08-27T18:02:25.376995Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has star
ted.
wordpress-db | 2026-08-27T18:02:25.906470Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ende
d.
wordpress-db | 2026-08-27T18:02:26.380785Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self
signed.
wordpress-db | 2026-08-27T18:02:26.380835Z 0 [System] [MY-013602] [Server] channel mysql_main configured
to support TLS. Encrypted connections are now supported for this channel.
wordpress-db | 2026-08-27T18:02:26.389384Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --
pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a differ
ent directory.
wordpress-db | 2026-08-27T18:02:26.429013Z 0 [System] [MY-011323] [Server] X Plugin ready for connections
. Socket: /var/run/mysqld/mysqld.sock
wordpress-db | 2026-08-27T18:02:26.429379Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for co
nnections. Version: '8.0.46' socket: '/var/run/mysqld/mysqld.sock' port: 0 MySQL Community Server - GPL
wordpress-db | 2026-08-27 18:02:26+00:00 [Note] [Entrypoint]: Temporary server started.
wordpress-db | 2026-08-27 18:02:26+00:00 [Note] [Entrypoint]: Temporary server started.

```

```
root@ip-172-31-30-131:~/wordpress-compose
[root@ip-172-31-30-131 wordpress-compose]# docker exec -it wordpress-db mysql -uwordpress -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| performance_schema |
| wordpress      |
+-----+
3 rows in set (0.01 sec)

mysql> USE wordpress;
Database changed
mysql> show tables;
Empty set (0.00 sec)

mysql> |
```

```
MINGW64:/c:/Users/brajk/Downloads
[root@ip-172-31-30-131 wordpress-compose]# docker compose down
[+] down 3/3
✓ Container wordpress          Removed          3.5s
✓ Container wordpress-db       Removed          1.4s
✓ Network wordpress-compose_default Removed          0.4s
[root@ip-172-31-30-131 wordpress-compose]# Connection to 54.221.137.85 closed by remote host.
Connection to 54.221.137.85 closed.
```

Step 29 — Stop WordPress

When finished:

`docker compose down`

This removes the containers but keeps named volumes.

Therefore your database data remains in:

`db_data`

and WordPress data remains in:

`wordpress_data`

Step 30 — Remove Everything Including Data

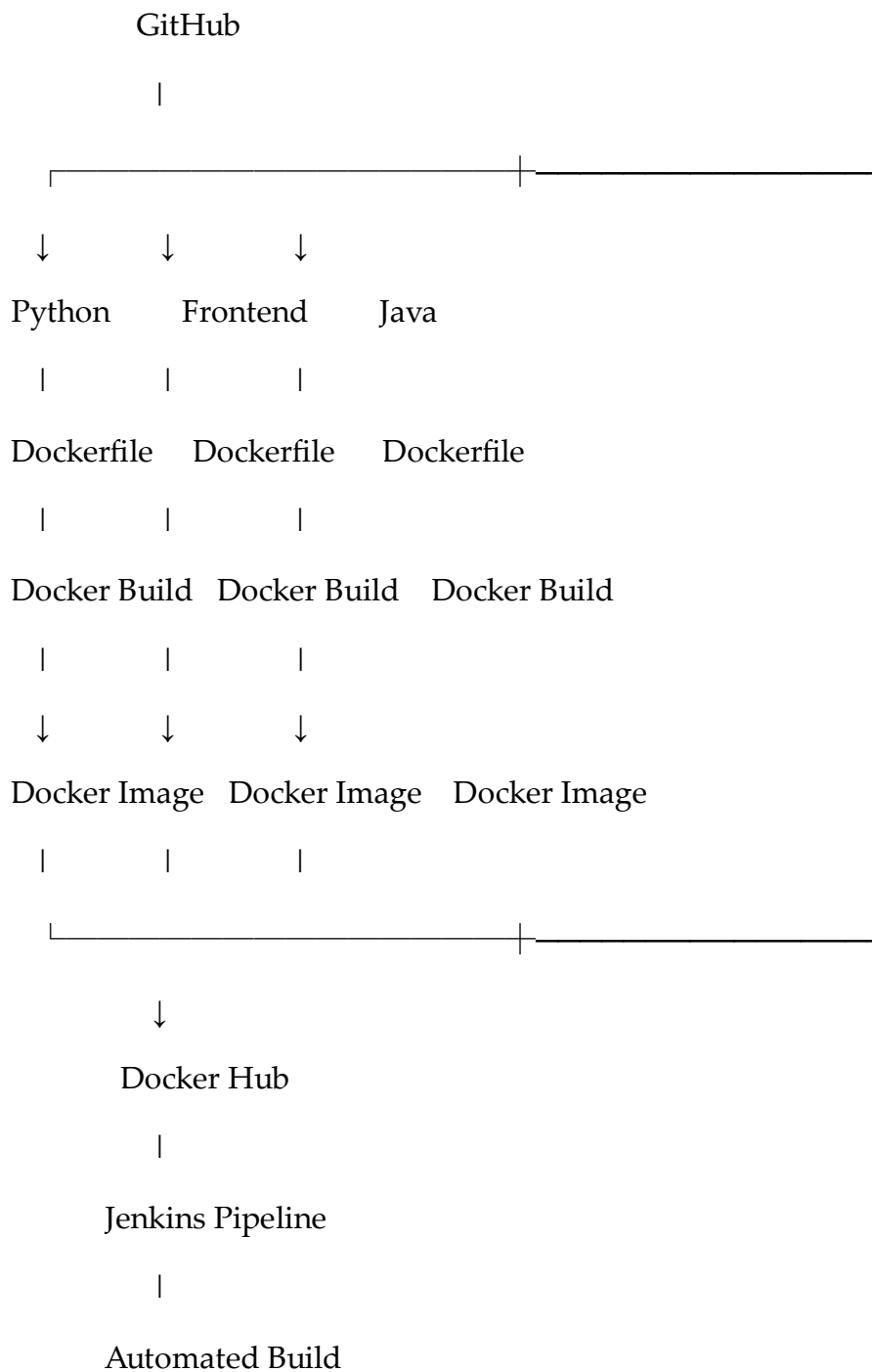
If you want to completely remove the assignment:

`docker compose down -v`

Be careful: `-v` deletes the named volumes and therefore your WordPress/MySQL data.

Final Assignment Structure

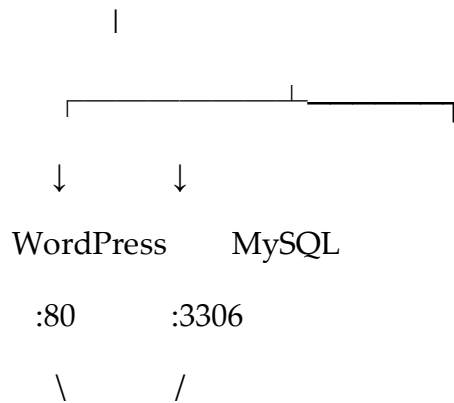
Your complete practical can be documented as:



|

Automated Push

Docker Compose



Docker Network

Commands Cheat Sheet

Python/Jenkins

```
git clone https://github.com/betawins/Python-app.git
```

```
docker build -t python-app:latest .
```

```
docker run -d -p 5000:5000 python-app:latest
```

```
docker tag python-app:latest USERNAME/python-app:latest
```

```
docker push USERNAME/python-app:latest
```

Frontend

```
docker build -t frontend:v1 .
```

```
docker run -d -p 8081:80 frontend:v1
```

```
docker tag frontend:v1 USERNAME/frontend:v1
```

```
docker push USERNAME/frontend:v1
```

Java

```
docker build -t java-app:v1 .
```

```
docker run -d -p 8082:8080 java-app:v1
```

```
docker tag java-app:v1 USERNAME/java-app:v1
```

```
docker push USERNAME/java-app:v1
```

Node.js

```
docker build -t node-app:v1 .
```

```
docker run -d -p 3000:3000 node-app:v1
```

```
docker tag node-app:v1 USERNAME/node-app:v1
```

```
docker push USERNAME/node-app:v1
```

WordPress + MySQL

```
docker compose config
```

```
docker compose up -d
```

```
docker compose ps
```

```
docker compose logs
```

```
docker compose down
```

Jenkins Pipeline Flow

GitHub



Git Clone



Docker Build



Docker Login



Docker Push



Docker Hub

One important point: for the frontend, Java, and Node.js tasks, the exact directory names and application ports in docker-tasks should be verified from the repository before you create the Dockerfiles. I don't want to give you a Dockerfile that assumes the wrong source directory or port. The repository is the correct starting point, but its current top-level listing isn't exposing the task subdirectories in the web result.

If you are doing this on the **Jenkins EC2 you just created**, the next practical step is **Task 1: create the Jenkins job for Python-app**. Start with the Docker/Jenkins prerequisites, then create the pipeline above.